

CHAPTER 12

Attitude control

12.1. Space story

A new type of thruster was the Improved Electrothermal Hydrazine Thruster (IMPEHT). In the previous spacecraft, monopropellant thrusters were used for attitude control during North/South station keeping and Electrothermal Hydrazine Thruster (EHTs) were used to produce the velocity change. IMPEHTs could be pulsed and were more efficient so they could be used for both, with the bonus of higher efficiency. When the first maneuver was done the attitude was jittery. After much analysis, this was determined to be due to helium bubbles from the blowdown-system pressurant mixed with the fuel. The more efficient IMPEHTs had smaller feed tubes so that some pulses were just hot helium. Attempts to “burp” the spacecraft proved unsuccessful. IMPEHTs were replaced by arc jets on subsequent spacecraft.

12.2. Introduction

This chapter discusses topics of control particularly relevant to spacecraft attitude control. Attitude control is a vast subject. In its most general form, it encompasses controlling the orientation of the spacecraft relative to a target orientation and the orientation of parts of the spacecraft relative to a target orientation or to the spacecraft itself. The target may be fixed in the inertial frame or fixed to a planet fixed frame. It may also be moving relative to either of those frames. When we talk about 3-axis control it means we want to control all three axes of the spacecraft with respect to the target orientation. Single-axis control (which is the case with a spinner) only requires control of one axis with respect to a target orientation. Fig. 12.1 shows a hierarchy of typical attitude control systems.

Attitude control systems can be passive or active. Active means that we take a measurement and compute a control action that is accomplished with an actuator. The computation can be analog or digital, although most modern spacecraft use digital control. A passive system uses the physics of the spacecraft and possibly a control actuator to perform control. For example, the dual-spin-turn involves turning on a momentum wheel mounted transversely to the axis of rotation. The momentum of the spacecraft is transferred to the wheel and the spacecraft nutates and then flips on its side. The nutation is usually damped through fuel slosh. This maneuver is entirely passive yet achieves the desired pointing. A gravity-gradient-stabilized spacecraft is another example of passive control. A libration damper damps excursions from the nominal attitude.

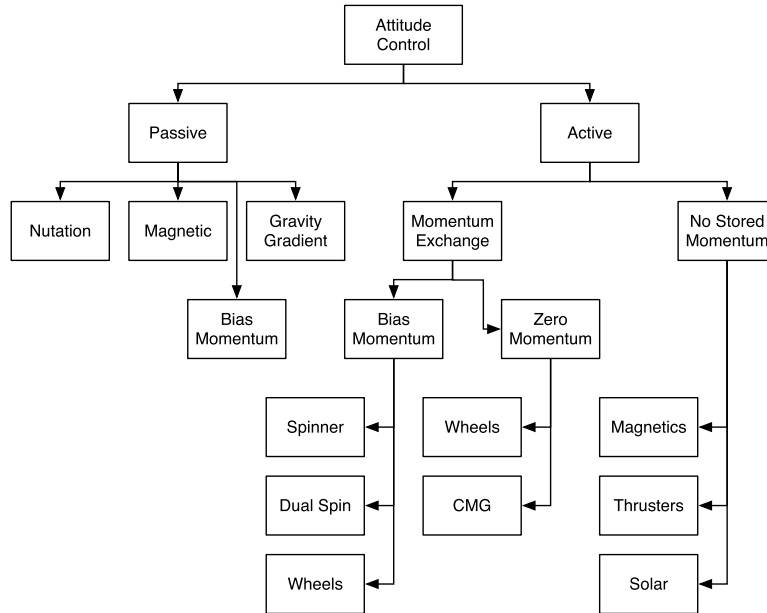


Figure 12.1 Attitude control system hierarchy. This diagram shows various types of attitude control systems.

The spacecraft dynamics, including gravity gradient, have oscillatory modes about pitch and roll.

Active control systems either can transfer momentum within the spacecraft or use external torques produced by thrusters, solar vanes, or magnetic torquers, to control the spacecraft. Momentum exchange is easy to understand. If the spacecraft is inertially fixed and you spin up a wheel the momentum going into the wheel comes from the spacecraft that rotates in the opposite direction to conserve momentum. This is the basis of the reaction wheel. No momentum is exchanged with the environment. Only energy is consumed. This is a very convenient method for reorienting a spacecraft or controlling the orientation in the face of cyclic disturbances. The momentum-less type reacts against the environment to produce torque. Magnetic torquers only use electricity and solar vanes use power to orient the vanes to produce the desired torque. Thrusters consume fuel. Momentum-exchange systems also require actuators to control inertially fixed torques that change the spacecraft momentum. These can be any of the actuators required by the momentum-less systems.

The rest of this chapter discusses attitude control systems in general and then covers some interesting examples of control systems.

12.3. Attitude control phases

Fig. 12.2 shows typical phases in the life of a spacecraft. The spacecraft starts on top of whatever vehicle delivers it to its initial orbit. The transfer vehicle may be 3-axis stabilized or spinning. If it is spinning the spacecraft needs to despin (if it is not a spinner) and remove any other angular rates. While it does this, it needs to maintain an orientation where there is Sun on the solar panels.

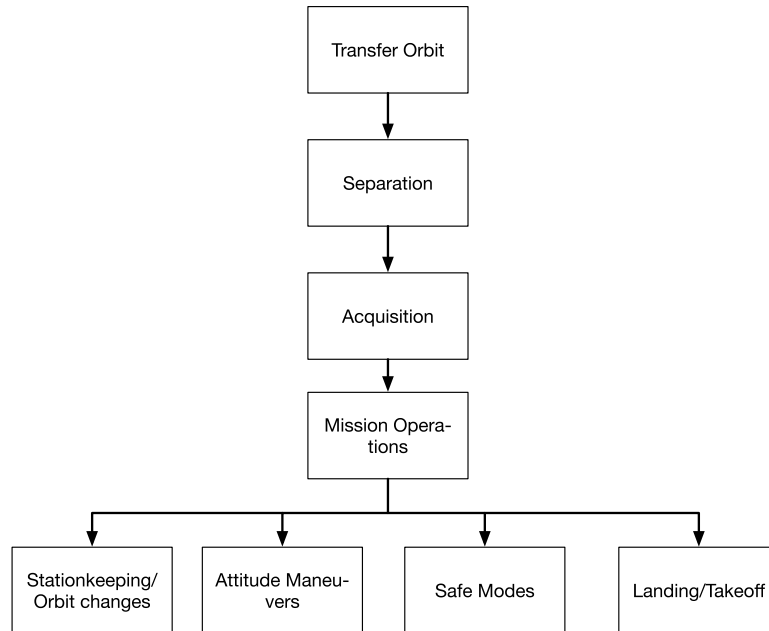


Figure 12.2 Attitude control phases.

After despin and detumbling the spacecraft needs to acquire its baseline attitude. This may require some kind of large-angle maneuver. Once it is in its nominal orientation the mission begins. For some spacecraft, the launch vehicle does not leave it in its final orbit and a series of propulsive maneuvers, lumped in with station keeping on the graph, must be done. This will require keeping the delta-V thrusters in a specific orientation. Station keeping may be required during the mission life. This is always necessary for geosynchronous spacecraft that must stay in their geosynchronous “box”.

Some spacecraft, for example, the NASA Hubble or James Webb Space Telescopes, must reorient to look at different targets. Others, like communications satellites, just point at the Earth. Some Earth-resources satellites need to orient sensors at particular points on the ground. This may involve large and rapid attitude maneuvers.

The final phase is known as a safe mode. Safe means there is power on the spacecraft. This leads to the term “Sun-safe mode” where the spacecraft spins around the Sun line with its solar panels pointed at the Sun. Not all spacecraft have safe modes.

12.4. Attitude control system

Fig. 12.3 shows a hierarchy of typical attitude control systems. In some systems, the blocks represent real blocks of software or analog hardware. In most systems, the block functions are distributed among many different pieces of software. The first step is to get information from the sensors. In some cases, this data can be used directly. For example, an Earth sensor will produce roll and yaw angles that can be used directly in the control algorithms, albeit with some filtering. In other cases, such as with a star tracker or camera, the positions of the stars on the focal plane must be converted into an attitude representation through further processing.

Some systems employ an estimator that is simply a filter that employs a model of the system. The model may be only of the sensor hardware or may be one of the spacecraft. The output is used to produce desired control accelerations. These must be converted to torque demand. For example, if the inputs are roll, pitch, and yaw and the outputs are angular accelerations, the desired torque can be computed from

$$T = Ia \quad (12.1)$$

where a is the control acceleration vector and I is the inertia matrix. If there are off-diagonal terms the torque will include contributions from one or more axis acceleration. The torque demand must be converted into actuator commands. In the case of reaction wheels, the 3-axis torque must be converted to 3 or more reaction-wheel torque commands. If thrusters are used these commands might be used to produce pulsewidth demands for 3 or more thrusters. This is done by the torque-distribution block. Finally, the actuator demands must be converted to a form suitable for use by the hardware in the actuator interface block.

A control system must always accommodate ground commands. These may be commands that go directly to hardware, in some cases bypassing the flight computer. In other cases, they may be additional axis-acceleration demands.

12.5. Single-axis control

It is instructive to design a controller for a single axis of a spacecraft. In many cases, such as in station keeping, this is common practice. As noted above, if our controller works with angular accelerations we can ignore cross-axis coupling. The single-axis controller is shown in Fig. 12.4. The control system is in series with the spacecraft dynamical

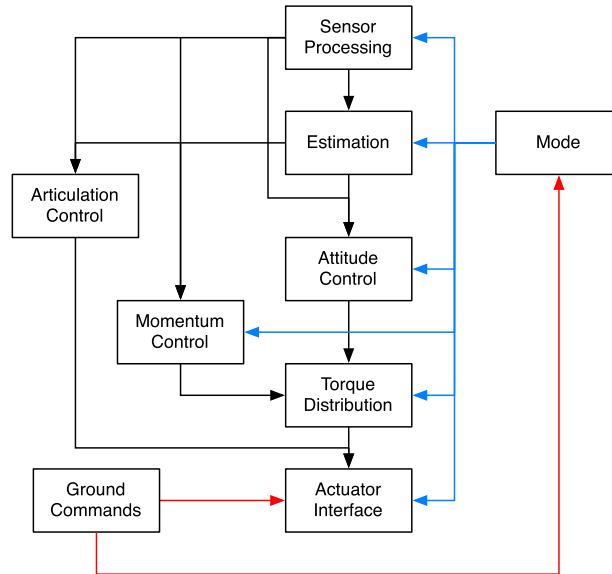


Figure 12.3 Attitude control system architecture.

model. We take measurements and feed them back, with a negative sign, to correct errors.

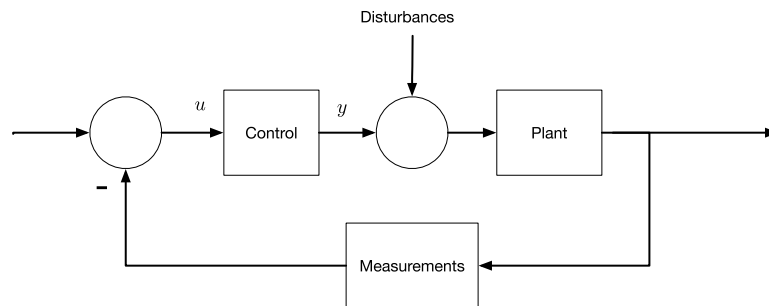


Figure 12.4 Control system.

If we model our system as a double integrator we get

$$\ddot{\theta} = a(t) \quad (12.2)$$

where θ is the angle and a is the angular acceleration. The Laplace transform is

$$s^2\theta = a(s) \quad (12.3)$$

where $s = j\omega$. $j = \sqrt{-1}$. With this model a constant acceleration will cause the angle to grow without bound. During a station-keeping maneuver this happens because the delta-V thrusters produce a constant disturbance acceleration.

Three possible controllers are, assuming we only measure angle (γ is an angle in this case)

$$\frac{\gamma}{u} = k \quad (12.4)$$

$$\frac{\gamma}{u} = k \left(1 + \frac{\tau_r s}{1 + \tau_f s} \right) \quad (12.5)$$

$$\frac{\gamma}{u} = k \left(1 + \frac{\tau_r s}{1 + \tau_f s} + \frac{1}{\tau_I s} \right) \quad (12.6)$$

The first is a proportional controller with a forward gain of k . The second adds a derivative, that is filtered. The derivative time constant is τ_r and the filter time constant is τ_f . Filtering the derivative term is done in industrial controllers because differentiation amplifies noise so you want to cut off the differentiation at some frequency. Another way of looking at it is that there is nothing you want to control at higher frequencies, thus everything about it is the noise. The third adds an integrator with a time constant of τ_I . This time constant is roughly how long the controller will take to integrate out the constant disturbance. Bode shows the Bode plots of the open-loop controllers in series with the open-loop system. The pure gain does not change the phase but just shifts the magnitude up. The derivative adds phase shift at the crossover frequency, where the magnitude is one. This shows that it is adding damping. The integral shows the gain increasing as s approaches zero, demonstrating integral action. Example 12.1 shows Bode and time-response plots.

With the first controller, the angle excursions will be limited but the spacecraft will oscillate. The second will damp the motion but there will be an offset. The third will eventually drive the error to zero.

It is instructive to look at how the open-loop response relates to the closed-loop response. The error that is the input to the control system is

$$e = u - \gamma \quad (12.7)$$

and

$$\gamma = CPe \quad (12.8)$$

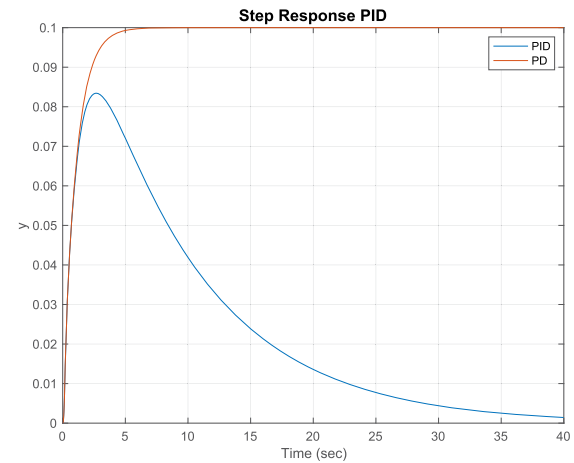
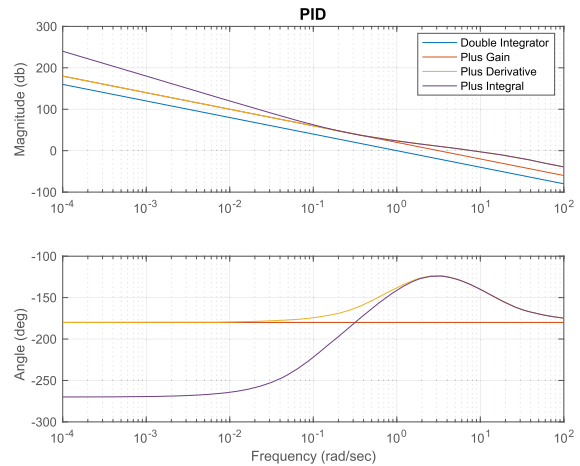
Solving for γ

$$\frac{\gamma}{u} = \frac{C}{\frac{1}{P} + C} \quad (12.9)$$

```

1 %% PID Control
2
3
4 w = logspace(-4,2);
5
6
7 % Double integrator
8 aS = [0 1; 0 0];
9 bS = [0; 1];
10 cS = [1 0];
11 dS = 0;
12 [m,p] = FResp(aS,bS,cS,dS,1,1,w);
13 m = 20*log10(m);
14
15 % Add gain
16 aP = 0;
17 bP = 0;
18 cP = 0;
19 dP = 10;
20 [a,b,c,d] = Series(aP,bP,cP,dP,aS,bS,cS,dS);
21 [m2,p2] = FResp(a,b,c,d,1,1,w);
22 m2 = 20*log10(m2);
23
24 % Add derivative
25 tauF = 0.1;
26 k = 10;
27 tauR = 1;
28 [aR,bR,cR,dR] = ND2SS(k*[tauR+tauF 1],[tauF 1]);
29 [a,b,c,d] = Series(aR,bR,cR,dR,aS,bS,cS,dS);
30 [m3,p3] = FResp(a,b,c,d,1,1,w);
31 m3 = 20*log10(m3);
32
33 % Add Integrator
34 tauI = 10;
35 f = k/tauI;
36 [aP,bP,cP,dP] = ND2SS(f*[tauI*(tauR+tauF) tauI+tauF 1],[tauF 1 0]);
37 [a,b,c,d] = Series(aP,bP,cP,dP,aS,bS,cS,dS);
38 [m4,p4] = FResp(a,b,c,d,1,1,w,'unwrap');
39 m4 = 20*log10(m4);
40
41
42 %% Plot
43 yL = {'Magnitude_(db)'; 'Angle_(deg)'};
44
45 Plot2D(w,[m;m2;m3;m4;p2;p3;p4], 'Frequency_(rad/sec)', yL, 'PID', ['xlog'; 'xlog'], {'[1 2 3 4]'; '[5 6 7 8]'});
46
47 legend('Double Integrator', 'Plus Gain', 'Plus Derivative', 'Plus Integral');
48
49
50
51 n = 2000;
52 dT = 0.02;
53 y = zeros(1,n);
54 u = 1;
55
56 [aC,bC,cC,dC] = CLoopS(aS,bS,cS,aP,bP,cP,dP);
57 [aC,bC] = C2DZOH(aC,bC,dT);
58
59 x = zeros(length(aC),1);
60 yI = zeros(1,n);
61 for k = 2:n
62     yI(1,k) = cC*x + dC*u;
63     x = aC*x + bC*u;
64 end
65
66 [aC,bC,cC,dC] = CLoopS(aS,bS,cS,aR,bR,cR,dR);
67 [aC,bC] = C2DZOH(aC,bC,dT);
68
69 x = zeros(length(aC),1);
70 yR = zeros(1,n);
71 for k = 2:n
72     yR(1,k) = cC*x + dC*u;
73     x = aC*x + bC*u;
74 end
75
76 [t,tL] = TimeLabl((0:n-1)*dT);
77
78 Plot2D(t,[yI;yR],tL,'y','Step_Response_PID');
79 legend('PID','PD')

```



Example 12.1: Bode and time-response plots.

If $C = 1$ given the double-integrator plant the resulting closed-loop system is

$$\frac{y}{u} = \frac{1}{s^2 + 1} \quad (12.10)$$

where $s = 0$; it is no longer ∞ . The denominator is that for an undamped oscillator. This is what the open-loop crossover frequency shows. The phase shift was 180 degrees at the crossover.

If $C = k(\zeta\sigma s + \sigma^2)$ we get

$$\frac{y}{u} = k \frac{\zeta\sigma s + \sigma^2}{s^2 + k\zeta\sigma s + k\sigma^2} \quad (12.11)$$

which is a damped oscillator.

12.6. Three-axis control

Each axis is controlled independently with a proportional-integral-differential controller. The basic system is shown in Fig. 12.5. The reference quaternion and measured quaternion are multiplied to produce small angles. The proportional-integral-differential controller produces a per-axis acceleration demand. This is multiplied by the inertia matrix to produce a torque demand. If we are using thrusters we convert this to a pulsewidth demand using linear programming, also known as the simplex algorithm. If we are using reaction wheels we use the pseudoinverse to convert torque demand to reaction-wheel torque demand. This is further converted into a voltage.

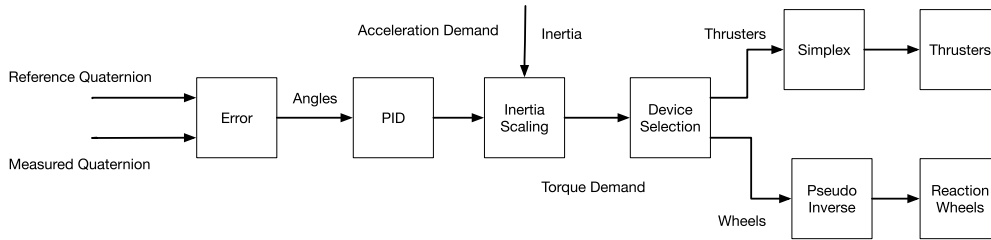


Figure 12.5 Attitude control system flow.

The rigid-body equations are

$$T = I\dot{\omega} + \omega^\times I\omega \quad (12.12)$$

If rates are small and the angles are small this reduces to

$$T = I\ddot{\theta} \quad (12.13)$$

If we define

$$T = Iu \quad (12.14)$$

then the axes are decoupled and the control problem is reduced to

$$u = \ddot{\theta} \quad (12.15)$$

or three decoupled double integrators. This is valid if the spacecraft angular rates are small and other modes, such as slosh or flexibility, are at frequencies much higher than the controller bandwidth. Once the control is computed the control torques are formed by the operation

$$T = Iu \quad (12.16)$$

which is a 3-by-3 matrix times a 3-by-1 vector multiplication. We can now design each axis control loop independently.

If we have two quaternions q_I and q_F that are the initial and final quaternions and we want to go from the first to the second we write

$$\Delta q = q_T^T \otimes q_I \quad (12.17)$$

As noted in the chapter on kinematics a quaternion is a unit vector and an angle about that unit vector. Therefore

$$\Delta q = \begin{bmatrix} \cos \frac{\phi}{2} \\ a_1 \sin \frac{\phi}{2} \\ a_2 \sin \frac{\phi}{2} \\ a_3 \sin \frac{\phi}{2} \end{bmatrix} \quad (12.18)$$

where a is the unit vector.

As an example, let the initial quaternion be

$$q_I = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (12.19)$$

and the target is

$$q_T = \begin{bmatrix} 0.8 \\ -0.6 \\ 0 \\ 0 \end{bmatrix} \quad (12.20)$$

We want to do a single-axis rotation. We do not need quaternions for this. The transpose of q_I is q_I and the product of q_I and q_T is q_T . Our unit vector is then

$$a = \frac{\begin{bmatrix} -0.6 \\ 0 \\ 0 \end{bmatrix}}{0.6} \quad (12.21)$$

or a vector aligned with the x -axis.

This works for arbitrary initial and target quaternions. Example 12.2 shows how it works for randomly selected quaternions. The magnitude of the angular rate is

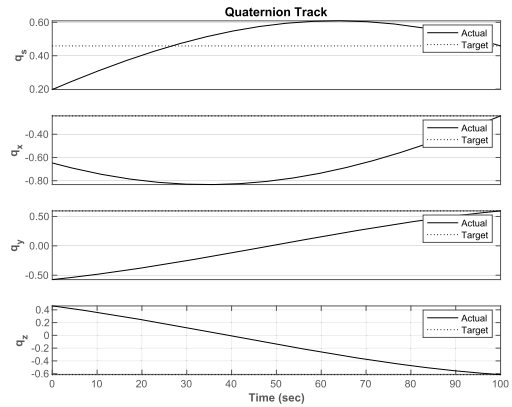
$$\omega = \frac{2 \cos^{-1} q(1)}{t} \quad (12.22)$$

where t is the desired maneuver duration.

```

1 qI = QRand;
2 qT = QRand;
3
4 deltaQ = QMult(QPose(qT), qI);
5 u = Unit(deltaQ(2:4));
6 angle = 2*acos(deltaQ(1));
7 t = linspace(0,100);
8 n = length(t);
9 omega = u*angle/t(n);
10 dT = t(2);
11 q = zeros(4,n);
12 q(:,1) = qI;
13
14 for k = 2:n
15     q(:,k) = RK4(@QDot, q(:,k-1), dT, t(k), omega);
16 end
17
18 qL = {'q_s' 'q_x' 'q_y' 'q_z'};
19
20 IL = {'Actual' 'Target'};
21 TimeHistory(t, [q; qT*ones(1,n)], qL, 'Quaternion_
    Track', ...
22     {[1 5] [2 6] [3 7] [4 8]}, {IL IL IL IL}, 1);
23
24
25 %% Right hand side;
26 function qDot = QDot(q, ~, omega)
27 qDot = QIToBDot(q, omega);
28 end

```



Example 12.2: Single-axis quaternion maneuver. The red lines are the target quaternion. The spacecraft reaches the target but does not stop at the target.

This is the most direct maneuver. Your spacecraft might have maneuver constraints, for example not having a star sensor to look at the Sun, in which case you would need to compute a less direct route.

This maneuver can be implemented in several ways. One is to break the maneuver into several chunks and use the PID to attain each delta angle. This makes your PID controller completely general-purpose as it is only dealing with the linear problem. An

alternative is to compute an angular rate, based on the momentum limits or torque limits of your control system. You would then track the desired angular rate and angle. Since the maneuver is single-axis you could also use a bang-zero-bang controller. That is, accelerate to the desired maneuver rate, coast, and then decelerate.

12.7. Gravity-gradient control

For many spacecraft, gravity-gradient control is sufficient to maintain Earth pointing. Torque is produced by the variation in gravity across the spacecraft. If the mass properties are correct, it will see restoring torques that will keep it nominally Earth pointing. Section 6.7 discusses gravity-gradient dynamics. As explained, a gravity-gradient-stabilized spacecraft has stiffness about all three axes, but no damping. Roll and yaw are coupled while pitch is decoupled. Thus we need roll/yaw and pitch damping. The dynamical equations are

$$T_x = I_x \ddot{\theta}_x + n \dot{\theta}_z (I_y - I_z - I_x) - 4n^2 \theta_x (I_z - I_y) \quad (12.23)$$

$$T_y = I_y \ddot{\theta}_y + 3n^2 (I_x - I_z) \theta_y \quad (12.24)$$

$$T_z = I_z \ddot{\theta}_z + n \dot{\theta}_x (I_z + I_x - I_y) - n^2 \theta_z (I_y - I_x) \quad (12.25)$$

Since we only want to damp angular rates, a rate gyro is a suitable sensor. The control torques are

$$T_x = -4\zeta n \sqrt{\frac{(I_y - I_z)}{I_x}} \dot{\theta}_x \quad (12.26)$$

$$T_y = -2\zeta n \sqrt{\frac{3(I_x - I_z)}{I_y}} \dot{\theta}_y \quad (12.27)$$

$$T_z = -2\zeta n \sqrt{\frac{(I_x - I_y)}{I_z}} \dot{\theta}_z \quad (12.28)$$

where ζ is the desired damping ratio. We can study this simply by forming the state-space matrices and adding the damping Example 12.3. The resulting eigenvalues show that we are close to achieving critical damping, with just rate feedback, along all three axes. We also show what linear quadratic output feedback achieves where the measurement matrix is just for the rates.

In both cases, the control is the desired angular acceleration. The simple rate controller works reasonably well. We could add gains to cancel the cross-axis terms to decouple the three axes completely. The full-state feedback controller has very low gains on an angle. There are small cross-axis gains. The full-state feedback controller has the same frequencies for all three axes. The controller increases the frequency of

```

1 %% Response of a gravity gradient system
2
3 iX = 0.2; iY = 0.4; iZ = 0.1;
4 n = sqrt(3.98600436e5/7000^3);
5 kX = 4*n^2*(iY-iZ)/iX;
6 kY = 3*n^2*(iX-iZ)/iY;
7 kZ = n^2*(iX-iZ)/iZ;
8 cX = n*(iY-iZ-iX)/iX;
9 cZ = n*(iZ+iX-iY)/iZ;
10 a = [ zeros(3,3) eye(3); ...
11       -kX 0 0 0 0 -cX; ...
12       0 -kY 0 0 0 0; ...
13       0 0 -kZ -cZ 0 0 ];
14 eig(a)
15 zeta = 0.7071;
16 u = -2*zeta*diag(sqrt([kX kY kZ]));
17 a(4:6,4:6) = a(4:6,4:6) + u;
18 eig(a)
19 c = [ zeros(3,3) eye(3) ];
20 b = [ zeros(3,3); eye(3) ];
21 k = QCR( a, b, eye(6), 1e10*eye(3) )
22 eig(a-b*k)

```

Undamped
-0.0000 ± 0.0028i
0.0000 ± 0.0010i
0.0000 ± 0.0009i

Damped
-0.0019 ± 0.0021i
-0.0007 ± 0.0007i
-0.0007 ± 0.0007i

Full State Feedback Gain Matrix
5.1260e-06 3.8755e-22 1.2362e-06 1.2305e-03 4.3687e-19 2.3552e-04 -
4.3567e-22 9.1663e-06 2.9010e-21 -9.7730e-21 3.1603e-03 6.7100e-19 -
1.4969e-06 1.6850e-21 8.8290e-06 2.3552e-04 5.1413e-19 2.9109e-03

Full State
-0.0025 ± 0.0028i
-0.0022 ± 0.0020i
-0.0022 ± 0.0022i

Example 12.3: Gravity gradient.

the pitch and yaw eigenvalues, that is it is adding stiffness to the system. Note that the cross-axis angular gains are of the same order of magnitude as the main diagonal terms. The cost of the control was adjusted so that the roll frequency is about the same as the rate damping case. You cannot just zero the angle gains, i.e., not measure angles. Interestingly, if you do you get four poles with zero imaginary parts and one complex pair.

One could also try output feedback assuming that we could measure pitch and roll and not yaw. Roll and pitch can be measured by an Earth sensor.

We have not specified how we would do the damping. Actuation could be done by rotational actuators, like reaction wheels, or by linear actuators aligned with the three axes and offset from the center-of-mass.

12.8. Nutation control

Nutation control is applicable to spinning spacecraft and those with momentum wheels. A step in angular acceleration in x will produce an offset in rate in z but not in x . A step in angular acceleration in z will produce an offset in rate in x but not in z . There is an undamped pole but the only zeros are at zero frequency.

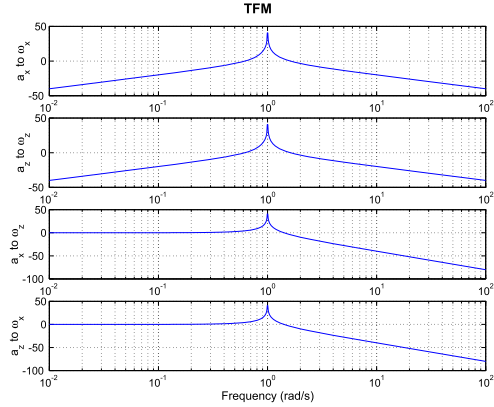
Damping can be provided by setting $a_x = -2\zeta k\omega_x$. The transfer function becomes

$$\begin{bmatrix} s + 2\zeta k & k \\ -k & s \end{bmatrix} \begin{bmatrix} \omega_x \\ \omega_z \end{bmatrix} = \begin{bmatrix} a_x \\ a_z \end{bmatrix} \quad (12.29)$$

```

1 k = 1;
2 a = [0 k;-k 0];
3 b = eye(2);
4 c = eye(2);
5 d = zeros(2,2);
6 w = logspace(-2,2,1000);
7 [m(1,:),p(1,:)] = Fresp(a,b,c,d,1,1,w);
8 [m(2,:),p(2,:)] = Fresp(a,b,c,d,2,2,w);
9 [m(3,:),p(3,:)] = Fresp(a,b,c,d,1,2,w);
10 [m(4,:),p(4,:)] = Fresp(a,b,c,d,2,1,w);
11 Plot2D(w,20*log10(m),'Frequency (rad/s)',...
12         {'a_x to omega_x' 'a_z to omega_z' 'a_x to omega_z' 'a_z to omega_x'}, 'TFM'
13         'xlog')
14 PrintFig(1,1,1,'NutationDampingTFM')

```



Example 12.4: Nutation dynamics.

The resulting solution is

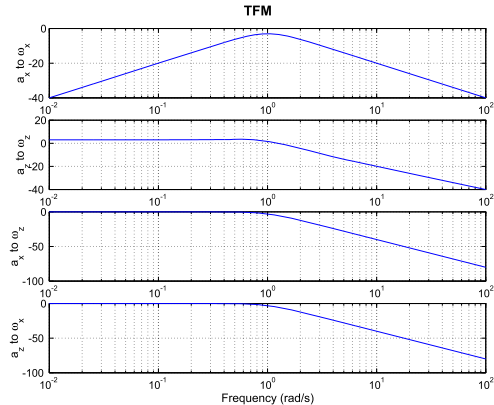
$$\begin{bmatrix} \omega_x \\ \omega_z \end{bmatrix} = \frac{\begin{bmatrix} s & -k \\ k & s + 2\zeta k \end{bmatrix}}{s^2 + 2\zeta ks + k^2} \begin{bmatrix} a_x \\ a_z \end{bmatrix} \quad (12.30)$$

The system is now critically damped at the expense of allowing a step response in the z -axis to produce a rate offset in z . An alternative is to set $a_x = -\zeta k \omega_x$ and $a_z = -\zeta k \omega_z$. This has the added advantage of tolerating a failure in either channel.

```

1 k = 1;
2 zeta = 0.7071;
3 a = [-2*zeta*k k;-k 0];
4 b = eye(2);
5 c = eye(2);
6 d = zeros(2,2);
7 w = logspace(-2,2,1000);
8 [m(1,:),p(1,:)] = Fresp(a,b,c,d,1,1,w);
9 [m(2,:),p(2,:)] = Fresp(a,b,c,d,2,2,w);
10 [m(3,:),p(3,:)] = Fresp(a,b,c,d,1,2,w);
11 [m(4,:),p(4,:)] = Fresp(a,b,c,d,2,1,w);
12 Plot2D(w,20*log10(m),'Frequency (rad/s)',...
13         {'a_x to omega_x' 'a_z to omega_z' 'a_x to omega_z' 'a_z to omega_x'}, 'TFM'
14         'xlog')
15 PrintFig(1,1,1,'NutationDampingTFM2')

```



Example 12.5: Nutation dynamics with a rate damper.

A step in angular acceleration in x will produce an offset in rate in z but not in x . A step in angular acceleration in z will produce an offset in rate in x but not in z .

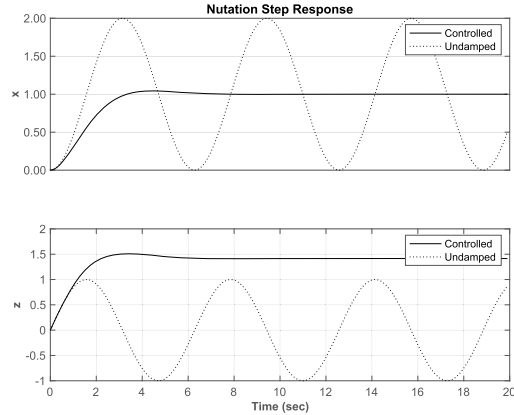
There is an undamped pole but the only zeros are at zero frequency. If the dual-channel approach is used there will be offsets in both axes.

Example 12.6 shows the step response of the damped and undamped system. Note the offset in z in the damped system.

```

1 k      = 1;
2 zeta   = 0.7071;
3 a      = [-2*zeta*k k;-k 0];
4 b      = eye(2);
5 c      = eye(2);
6 d      = zeros(2,2);
7 g      = statespace(a,b,c,d);
8 y      = Step(g,2,0.1,200);
9
10 k      = 1;
11 zeta   = 0;
12 a      = [2*zeta*k k;-k 0];
13 g      = statespace(a,b,c,d);
14 [y2,x,t] = Step(g,2,0.1,200);
15 IL      = {'Controlled' 'Undamped'};
16
17 TimeHistory(t,[y;y2], {'x' 'z'},...
18             'Nutation_Step_Response',[1 3] [2 4]),{
19             IL,IL},1)

```



Example 12.6: Step response.

12.9. Momentum-bias Earth-pointing control

The roll/yaw and pitch dynamics can be decoupled for analysis of normal operations. The pitch dynamics are a double integrator. The roll dynamics are a coupled fourth-order plant. There are two pure imaginary pole pairs. The first at orbit rate is due to the kinematics, the second is the nutation mode, and is due to the bias momentum. The main diagonal channels each have a pure imaginary zero pair. This zero pair is located between the orbit rate and nutation mode providing 180 degrees of phase shift and making the control problem easier.

Disturbances at orbit rate, or the nutation frequency, will cause uncontrolled growth in the attitude errors. Since many of the disturbance sources are at orbit rate, it is necessary to add an automatic control system. The nutation pole causes large oscillatory responses whenever a sudden torque is applied to the spacecraft.

The high- and low-frequency limits are of interest. At high frequencies as s approaches infinity, the system becomes a double integration. At low frequencies, the orbit-rate pole pair dominates.

The attitude kinematics come from the small-angle approximation

$$\omega = \dot{\theta} + (1 - \theta^\times)v \quad (12.31)$$

where v is the orbit rate vector. The dynamical equations are

$$I\dot{\omega} + \omega^\times(I\omega + h_W) + \dot{h}_W = T \quad (12.32)$$

For the purpose of analyzing the roll/yaw dynamics assume that h_W is constant. The orbit-rate vector is $[0 \quad -\omega_o \quad 0]^T$. Assume also that the inertia matrix is diagonal. Linearizing the dynamics, and coupling with the kinematics gives the transfer-function matrix

$$\begin{bmatrix} \theta_x(s) \\ \theta_y(s) \end{bmatrix} = \frac{\begin{bmatrix} I_z s^2 + \omega_o h_z & (\omega_o I_x - h_x)s \\ -(\omega_o I_z - h_z)s & I_x s^2 + \omega_o h_x \end{bmatrix}}{I_x I_z (s^2 + \omega_o^2) \left(s^2 + \left(\frac{h_x h_z}{I_x I_z} \right) \right)} \begin{bmatrix} T_x/I_x \\ T_x/I_x \end{bmatrix} \quad (12.33)$$

$$h_x = h_w + \omega_o(I_y - I_z)$$

$$h_z = h_w + \omega_o(I_y - I_x)$$

The roll/yaw equations and the equation of the damper wheel are

$$\begin{aligned} I_x \dot{\omega}_x + \omega_y \omega_z (I_z - I_y) - \omega_z h_w + J(\dot{\Omega} + \dot{\omega}_x) &= T_x \\ I_z \dot{\omega}_z + \omega_y \omega_x (I_y - I_x) - \omega_y (\Omega + \omega) + \omega_x h_w &= T_z \\ J(\dot{\Omega} + \dot{\omega}_x) &= -D\Omega \end{aligned} \quad (12.34)$$

where J is the inertia of the damper wheel and D is the damping coefficient. If we lump J with I_x we can simplify the above to

$$\begin{aligned} \dot{\omega}_x + k_x \omega_z + \epsilon \dot{\Omega} &= 0 \\ \dot{\omega}_z + k_z \omega_x - \gamma \dot{\Omega} &= 0 \\ \dot{\Omega} + \dot{\omega}_x &= \sigma \Omega \end{aligned} \quad (12.35)$$

where

$$k_x = \frac{\omega_y(I_z - I_y) - h_w}{I_x} \quad (12.36)$$

$$k_z = \frac{\omega_y(I_y - I_x) + h_w}{I_z} \quad (12.37)$$

$$\gamma = \frac{\omega_y J}{I_z} \quad (12.38)$$

$$\sigma = \frac{D}{J} \quad (12.39)$$

$$\epsilon = \frac{J}{I_x} \quad (12.40)$$

The characteristic equation can be written in Evans form

$$\frac{s^2 - \left(\frac{k_x(k_z + \gamma)}{1 - \epsilon} \right)}{s^2 - k_x k_z} = -\frac{\sigma}{1 - \epsilon} \quad (12.41)$$

When $\sigma = 0$ the numerator polynomial gives the poles of the system. When $\sigma = \infty$ the denominator gives the poles of the system. Hence, the open-loop zeros are given by the denominator and the open-loop poles by the numerator. The $s = 0$ pole is the damper-wheel pole and the quadratic terms give the roll/yaw poles. For stability, the open-loop zeros must be between the closed-loop poles. These equations are only valid for $J \neq 0$.

$$\begin{bmatrix} \theta_x(s) \\ \theta_y(s) \end{bmatrix} = \frac{\begin{bmatrix} 1/I_x & 0 \\ 0 & 1/I_z \end{bmatrix}}{s^2} \begin{bmatrix} T_x \\ T_z \end{bmatrix} \quad (12.42)$$

As is evident from the transfer-function matrix, high-bandwidth controllers can ignore the orbit rate and nutational dynamics. At low frequencies the plant becomes

$$\begin{bmatrix} \theta_x(s) \\ \theta_y(s) \end{bmatrix} = \frac{\begin{bmatrix} \omega_o & (\omega_o I_x/h_x - 1)s \\ (1 - \omega_o I_z/h_z)s & \omega_o \end{bmatrix}}{s^2 + \omega_o^2} \begin{bmatrix} T_x/h_x \\ T_z/h_z \end{bmatrix} \quad (12.43)$$

The diagonal terms are small and the dynamics are dominated by crosscoupling torques. Controllers concerned with attenuating low frequency (near orbit rate) can ignore the nutation mode.

Since there will always be an external bias torque on the spacecraft due to external disturbances, which causes the wheels to accumulate momentum over time, the wheels must be unloaded. To unload a wheel is to reduce the average speed back to its nominal value. This can be done with any method that also produces an external torque, i.e., magnetic torquers, reflective surfaces, aerodynamic surfaces, or thrusters. An alternative is to partition the control effort so that the inertially fixed disturbance compensation is performed by thrusters or torquers, and only the cyclic disturbances are controlled by the wheels. This can be done with a complementary filter pair – a matched low- and high-pass filter pair such as

$$\frac{u_{\text{wheel}}}{u} = \frac{s}{s + a} \quad \frac{u_{\text{thruster}}}{u} = \frac{a}{s + a} \quad (12.44)$$

where a is the break frequency and $s = j\omega$. Choosing a lower break frequency will mean the wheels store more momentum, while choosing a higher frequency will cause the thrusters to fire more often. The value must be selected so that the wheels never saturate over the necessary period.

Reaction wheels must pass through zero wheel speed; this presents a problem because the internal friction torques have a discontinuity, shown in Fig. 12.6. This is highly

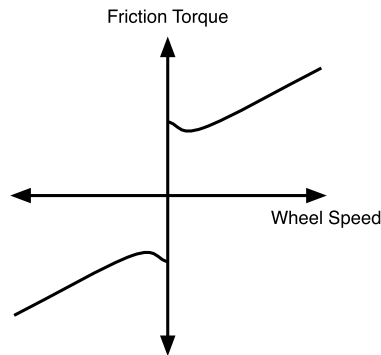


Figure 12.6 Friction-torque diagram.

nonlinear and not amenable to linearization about the zero wheel speed! The wheel can also get stuck at zero speed. One standard approach is to use triangle-wave dither in the voltage input to the wheel motor. In this technique, a triangle voltage wave is added to the desired voltage command. The command to the wheels is the sum of the triangle wave and the commands. The triangle-wave amplitude is a few least-significant bits. The effect of dither and other methods of improving control resolution is shown in Example 12.7. A PD controller is designed using the toolbox function `PDDesign` and discretized using a zero-order hold (`C2DZOH`). The ideal step response is shown in blue and labeled “no minimum pulsewidth” in the legend, along with three methods of handling finite pulsewidth.

The best result is obtained by using a different output period from the period used for control. For example, the algorithm might run at 2 Hz but the output is only sampled at 0.25 Hz. Triangle dither also works well. Another approach is to embed the wheel in a tachometer loop where the outer loop commands wheel-speed changes and the inner loop tries to force the wheel speed to track the desired speed changes. This tachometer loop is in addition to the current feedback loop, used in most reaction wheels to control the back EMF.

12.10. Mixed control

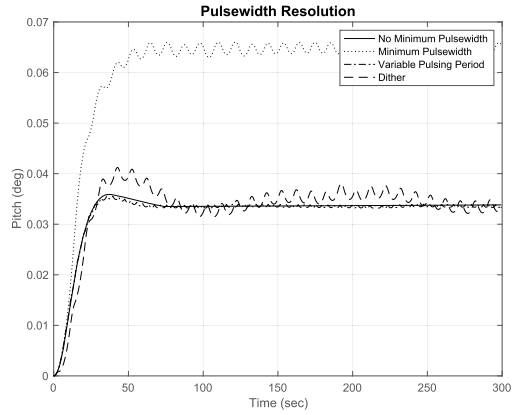
The spacecraft can be controlled using fewer than three wheels, if the magnetic torquers are used to supplement the wheels. The relationship between torque and dipole is

$$T = m \times b = -b \times m \quad (12.45)$$

```

1  inr = 2000; dT = 0.5;
2  minPw = 20; mDither = 25;% milliseconds
3  nPulse = 8; wDither = 30;
4  zeta = 0.7071; wN = 0.1; wD = 1.0;
5  [a,b,c,d] = PDDesign(zeta,wN,wD,inr);
6  [aN,bN,cN,dN] = CButter(2,1);
7  [aC,bC,cC,dC] = Series(a,b,c,d,aN,bN,cN,dN);
8  [aC,bC] = C2DZOH(aC,bC,dT);
9  aP = [0,1;0 0];
10 bP = [0;1]/inr;
11 [aP,bP] = C2DZOH(aP,bP,dT/500);
12
13 nSim = 600; uStep = 0.01;
14 yPlot = zeros(4,nSim); uPlot = yPlot;
15 dither = mDither*TriangleWave(linspace(0,2*pi*
    wDither,nSim));
16 uTorque = 0.5; nPW = 1000*dT;
17
18 for j = 1:4
19     x = zeros(2,1);
20     xC = zeros(3,1);
21     kPulse = 0;
22     for k = 1:nSim
23
24         yPlot(j,k) = x(1);
25         uC = cC*xC + dC*x(1);
26         xC = aC*xC + bC*x(1);
27         pulsewidth = dT*abs(uC/uTorque)*1000;
28
29         if( j == 3 )
30             kPulse = kPulse + 1;
31             if( kPulse < nPulse )
32                 pulsewidth = 0;
33             else
34                 pulsewidth = nPulse*pulsewidth;
35                 kPulse = 0;
36             end
37         elseif( j == 4 )
38             pulsewidth = pulsewidth + dither(k);
39         end
40
41         if( (j > 1) & (pulsewidth < minPw) )
42             pulsewidth = 0;
43         end
44
45         uPlot(j,k) = abs(pulsewidth);
46         nPulsewidth = floor(pulsewidth);
47         uCA = [uTorque*sign(uC)*ones(1,nPulsewidth),
48             zeros(1,nPW-nPulsewidth)];
49         for i = 1:nPW
50             x = aP*x + bP*(uStep - uCA(i));
51         end
52     end
53
54 Plot2D((0:(nSim-1))*dT, yPlot*180/pi, ...,
55     'Time_(sec)', 'Pitch_(deg)', ...,
56     'Pulsewidth_Resolution');
57 legend('No_Minimum_Pulsewidth', ...,
58     'Minimum_Pulsewidth', ...,
59     'Variable_Pulsing_Period', ...,
60     'Dither');
61 PrintFig(1,1,1,'Pulsewidth')

```



Example 12.7: Control resolution example showing multiple ways of handling finite pulsewidth.

or

$$T = \begin{bmatrix} 0 & b_z & -b_y \\ -b_z & 0 & b_x \\ b_y & -b_x & 0 \end{bmatrix} m \quad (12.46)$$

The torquers can produce an exact two-axis torque

$$m = \text{pinv}(a(k, :)) T(k) \quad (12.47)$$

where $b(k, :)$ are the rows of T that you want to produce and pinv is the pseudoinverse and $a = -b$. This allows one reaction wheel to be operated with the three magnetic torquers to produce a given 3-axis control torque.

A single-axis torque can be realized with two torquers. For example, write the equation for the y -axis.

$$T_y = m_z b_x - m_x b_z \quad (12.48)$$

Then,

$$m_z = \frac{T_y + m_x b_z}{b_x} \quad (12.49)$$

Find the dipoles that minimizes

$$m_x^2 + m_z^2 \quad (12.50)$$

This is found by replacing m_z in the quadratic, taking the derivative and solving for the dipole

$$m_x = -\frac{b_z T_y}{b_x^2 + b_z^2} \quad (12.51)$$

12.11. Magnetic-torquer-only control

Magnetic torquers are simple, and relatively inexpensive attitude control devices. The following sections discuss ways to use magnetic torquers.

12.11.1 BDot

BDot, which stands for magnetic-field derivative, is a way of damping angular rates without a gyro. The magnetic field can be measured by a magnetometer in the body frame. Let the transformation matrix from the inertial to the body frame be A . The transformation is

$$b_b = A b_i \quad (12.52)$$

The derivative (assuming that the field is not changing in the inertial frame) is

$$\dot{A} = A \omega^\times \quad (12.53)$$

Therefore

$$\dot{b}_b = \omega^\times b_b \quad (12.54)$$

The torque due to a magnetic torquer is

$$T = M^\times b_b \quad (12.55)$$

For a damper we want the torque to be

$$T = -k\omega \quad (12.56)$$

We can make a controller of the form

$$M = k\omega \dot{b} \quad (12.57)$$

which results in a controller

$$T = -kb_b^\times b_b^\times \omega \quad (12.58)$$

The controller in the s -plane is

$$M = skb \quad (12.59)$$

where s is the derivative, b is the magnetic field, and k is the gain. It is a good idea to provide some filtering. A first-order filter can be added

$$M = \frac{skb}{\tau_f s + 1} \quad (12.60)$$

This can be made into a discrete-time controller by using the first different operator

$$s = \frac{1 - z^{-1}}{\tau} \quad (12.61)$$

where τ is the time step. The resulting derivative is

$$\dot{b}_k = \frac{\tau \dot{b}_{k-1} + b_k - b_{k-1}}{\tau + \tau_f} \quad (12.62)$$

Example 12.8 shows the damping response to an initial body rate.

12.12. Low-bandwidth small-angle control

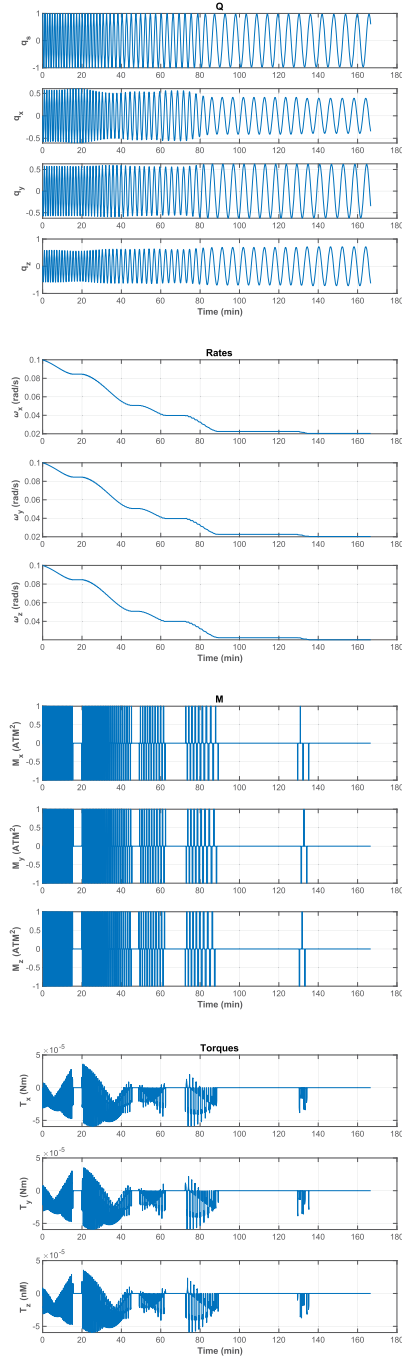
We will design a controller for a momentum-bias satellite in low-Earth orbit. It is Earth pointing. The control system is shown in Fig. 12.7. The momentum wheel controls pitch using a PD controller. Roll and yaw are controlled using state feedback. The angular momentum is

$$H = A(I\omega + uJ(\Omega + \omega)) \quad (12.63)$$

```

1 secInDay      = 86400;
2
3 % Parameters
4 inertia = eye(3);
5 dT = 1;
6 gain = 10;
7 mMax = 1;
8 mMin = 1e-6;
9 el = [7000;pi/3;0;0;0;0];
10 jD0 = Date2JD([2024 4 4]);
11 tEnd = 10000; % Duration of the simulation
12 tauFilter    = 10; % s
13 omega = [0.1;0.1;0.1];
14 q = [1;0;0;0];
15
16 %% End of User Inputs
17 n = ceil(tEnd/dT); % Time steps
18 t = linspace(0,tEnd,n); % Time array
19 jD = jD0 + t/secInDay; % Julian date array
20 dRHS.inertia = inertia;
21 dRHS.mu = 3.98600436e5;
22 % Orbit
23 [r,v] = E12RV(el);
24 % Plotting arrays
25 xP = zeros(19,n);
26 % Simulation loop
27 x = [r;v;q;omega];
28 % For the control
29 bBodyOld = QForm(x(7:10),BDipole(x(1:3),jD(1)));
30 bDotOld = [0;0;0];
31 for k = 1:n
32     % ECI to body quaternion
33     qECIToBody = x(7:10);
34     % Magnetic field
35     bECI = BDipole(x(1:3),jD(k));
36     bBody = QForm(qECIToBody,bECI);
37     % Low pass filter
38     deltaB = bBody-bBodyOld;
39     bDot = (dT*bDotOld + deltaB)/(tauFilter +
40         dT);
41     bDotOld = bDot;
42     bBodyOld = bBody;
43     dipole = gain*bDot;
44     j = find(abs(dipole)<mMin);
45     dipole(j) = 0;
46     dipole = -mMax*sign(dipole);
47     dRHS.torque = Cross(dipole,bBody);
48
49 % Plotting storage
50 xP(:,k) = [x;dipole;dRHS.torque];
51
52 % Propagate the state
53 x = RK4(@RHS,x,dT,0,dRHS);
54 end
55
56 % Plot labels
57 yL = ['r_x_0(km)' 'r_y_0(km)' 'r_z_0(km)' ...
58     'v_x_0(km/s)' 'v_y_0(km/s)' 'v_z_0(km/s)' ...
59     'q_s' 'q_u' 'q_v' 'q_w' ...
60     '\omega_x_0(rad/s)' '\omega_y_0(rad/s)' '\omega_z_0(
61         rad/s)';...
62     'M_x_0(ATM^2)' 'M_y_0(ATM^2)' 'M_z_0(ATM^2)' ...
63     'T_x_0(Nm)' 'T_y_0(Nm)' 'T_z_0(Nm)'];
64 % Plot
65 k = 7:10;
66 TimeHistory(t,xP(k,:),yL(k),'Q'); k = 11:13;
67 TimeHistory(t,xP(k,:),yL(k),'Rates'); k = 14:16;
68 TimeHistory(t,xP(k,:),yL(k),'M'); k = 17:19;
69 TimeHistory(t,xP(k,:),yL(k),'Torques');
70 Figui
71
72 %% Right hand side function
73 function xDot = RHS(x,~,d)
74
75 r = x(1:3);
76 v = x(4:6);
77 q = x(7:10);
78 omega = x(11:13);
79
80 omegaDot = d.inertia\(d.torque - Cross(omega,d.
81     inertia*omega));
82 xDot = [v;-d.mu*r/Mag(r)^3;QToBDot(q,omega);
83     omegaDot];
84 end

```



Example 12.8: BDot simulation.

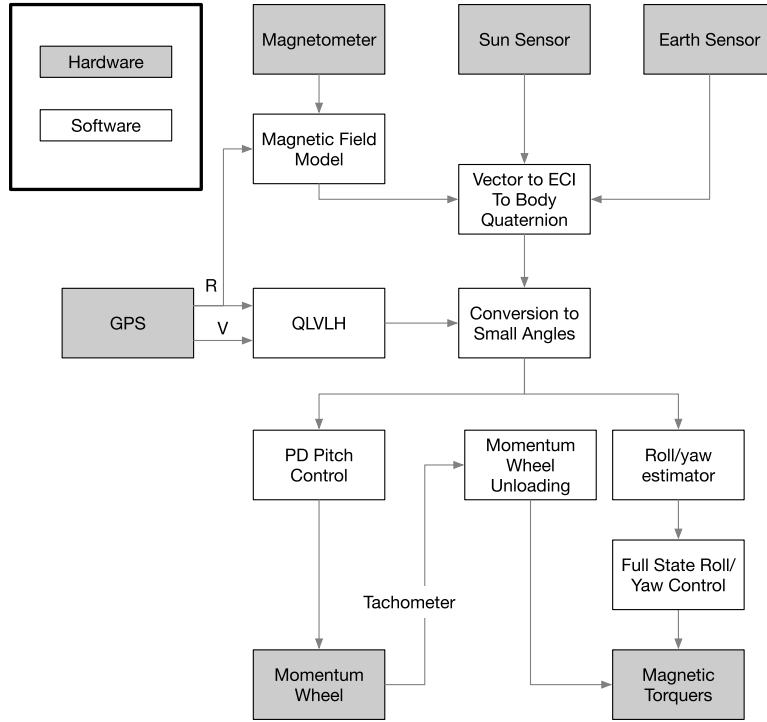


Figure 12.7 Low-bandwidth control system.

where u is the wheel spin-rate unit vector and J is the momentum of inertia about that axis. Ω is the angular rate of the wheel relative to the core spacecraft. The equations of motion are found by taking the time derivative in the inertial frame.

$$T = I\dot{\omega} + uJ(\dot{\Omega} + \dot{\omega}) + \omega^\times (I\omega + uJ(\Omega + \omega)) \quad (12.64)$$

$$T_w = J(\dot{\Omega} + \dot{\omega}) \quad (12.65)$$

These can be further simplified,

$$T = I\dot{\omega} + uT_w + \omega^\times h_w \quad (12.66)$$

$$T_w = J(\dot{\Omega} + \dot{\omega}) \quad (12.67)$$

If the angles are small, the transformation matrix from LVLH to body is

$$A = \begin{bmatrix} 1 & \theta_z & -\theta_y \\ -\theta_z & 1 & \theta_x \\ \theta_y & -\theta_x & 1 \end{bmatrix} \quad (12.68)$$

θ_x about x , θ_y about y , and θ_z about z . The angles and signs can be found by inspection by looking at Fig. 12.8.

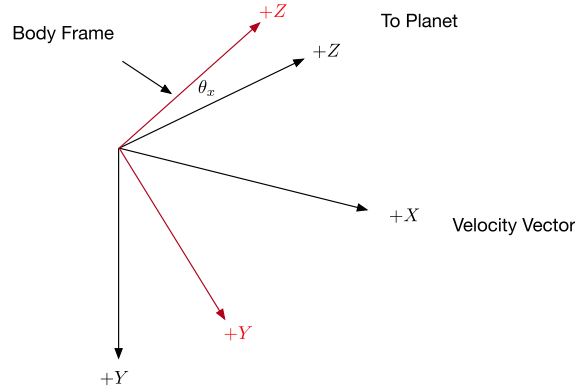


Figure 12.8 Coordinate transformations from LVLH to body with small angles. This diagram shows the roll angle, α . The body axes are in red.

Assume that the spacecraft inertial rotation rate is ω_o about $-Y$, then the angular rates in the body frame are

$$\omega = \begin{bmatrix} \dot{\theta}_x \\ \dot{\theta}_y \\ \dot{\theta}_z \end{bmatrix} + \begin{bmatrix} 1 & \theta_z & -\theta_y \\ -\theta_z & 1 & \theta_x \\ \theta_y & -\theta_x & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -\omega_o \\ 0 \end{bmatrix} \quad (12.69)$$

The dynamical equations are

$$I_x \dot{\omega}_x + h_w \omega_z = T_x \quad (12.70)$$

$$I_y \dot{\omega}_y = T_y - T_w \quad (12.71)$$

$$I_z \dot{\omega}_z - h_w \omega_x = T_z \quad (12.72)$$

The system is linear. The state equations are

$$\begin{bmatrix} \dot{\theta}_x \\ \dot{\theta}_y \\ \dot{\theta}_z \\ \dot{\omega}_x \\ \dot{\omega}_y \\ \dot{\omega}_z \end{bmatrix} = \begin{bmatrix} 0 & 0 & -\omega_o & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ \omega_o & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & \frac{h_w}{I_x} \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -\frac{h_w}{I_z} & 0 & 0 \end{bmatrix} \begin{bmatrix} \theta_x \\ \theta_y \\ \theta_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \quad (12.73)$$

$$+ \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \frac{1}{I_x} & 0 & 0 \\ 0 & \frac{1}{I_y} & 0 \\ 0 & 0 & \frac{1}{I_z} \end{bmatrix} \begin{bmatrix} T_x \\ T_y - T_w \\ T_z \end{bmatrix}$$

The roll/yaw state equations are

$$\begin{bmatrix} \dot{\theta}_x \\ \dot{\theta}_z \\ \dot{\omega}_x \\ \dot{\omega}_z \end{bmatrix} = \begin{bmatrix} 0 & -\omega_o & 1 & 0 \\ \omega_o & 0 & 0 & 1 \\ 0 & 0 & 0 & \frac{h_w}{I_x} \\ 0 & 0 & -\frac{h_w}{I_z} & 0 \end{bmatrix} \begin{bmatrix} \theta_x \\ \theta_z \\ \omega_x \\ \omega_z \end{bmatrix} \quad (12.74)$$

$$+ \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ \frac{1}{I_x} & 0 \\ 0 & \frac{1}{I_z} \end{bmatrix} \begin{bmatrix} T_x \\ T_z \end{bmatrix}$$

The controller uses full-state feedback and is

$$T = -kx \quad (12.75)$$

Generally, angular rates are not measured directly. An easy way to find the rates, given the model, is to make a full-state estimator. The estimator will be of the form

$$\dot{x}_E = Ax_E + k_E(\gamma - Cx_E) \quad (12.76)$$

where k_E is the optimal estimator gain, γ is the measurement vector, and C is the measurement matrix

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \quad (12.77)$$

The measurement vector is

$$\gamma = \begin{bmatrix} \theta_x \\ \theta_z \end{bmatrix} \quad (12.78)$$

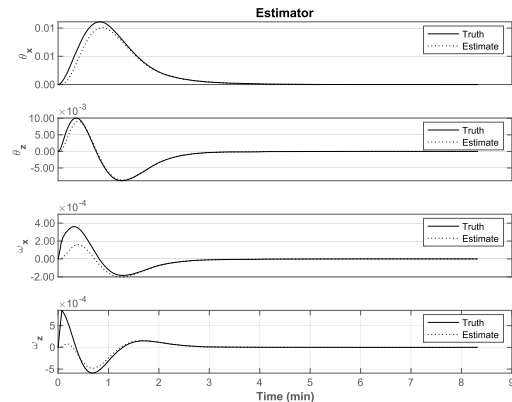
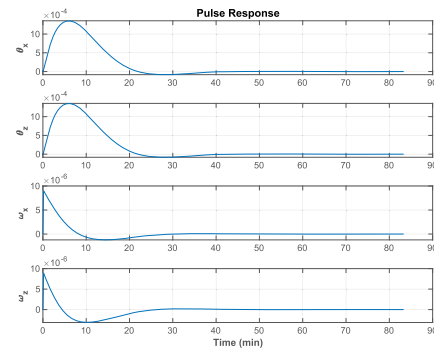
The parameters for the satellite, a 3U CubeSat, are given in Table 12.1.

Example 12.9 shows the control response. The script allows you to explore different control options. Note that the estimator response tracks the truth values very well, even though the estimator does not include the control torque. You can try stabilizing the system with only one control torque. While you can stabilize it, you cannot control both axes.

```

1 oneTorque = false; % Pick option to use one
  torque
2
3 omegaO    = 2*pi/(60*90);
4 hW        = 0.0005;
5
6 iX = 0.025; iY = .025; iZ = 0.005;
7
8 % Full system
9 a = [0      -omegaO 1      0;...
10     omegaO 0      0      1;...
11     0      0      0      hW/iX;...
12     0      0      -hW/iZ 0];
13
14 b = [0 0; 0 0; 1/iX 0; 0 1/iZ];
15 c = [1 0 0 0; 0 1 0 0];
16
17 if ( oneTorque )
18     bC = [0;0;0;1/iZ]; %ok<UNRCH>
19     r = 1e8;
20 else
21     bC = b;
22     r = 1e10*eye(2);
23 end
24
25 disp('Open Loop Eigenvalues')
26 eig(a)
27
28 kC = QCR(a,bC,diag([1 1 0.1 0.1]),r);
29
30 kE = QCE( a, c, 1e-10*eye(2), 0.1*eye(2), b );
31
32 % Controller gain
33 disp(kC);
34
35 % Estimator gain
36 disp(kE);
37
38 % Closed loop eigenvalues
39 aCL = a-bC*kC;
40 disp('Closed Loop Eigenvalues')
41 eig(aCL)
42
43 % Pulse response
44 dT = 0.5;
45 jP = 10;
46 [aD,bD] = C2DZOH(a,b,dT);
47 [~,bCD] = C2DZOH(a,bC,dT);
48 tP = [1e-6;1e-6];
49
50 n = 1000;
51 x = zeros(4,n);
52 xE = zeros(4,n);
53 for j = 2:n
54     xEJM1 = x(:,j-1);
55     %xEJM1 = x(:,j-1);
56     if ( j <= jP )
57         x(:,j) = aD*x(:,j-1) - bC*kC*xEJM1 + bD*tP;
58     else
59         x(:,j) = aD*x(:,j-1) - bC*kC*xEJM1;
60     end
61     xE(:,j) = aD*xEJM1 + kE*(x(1:2,j-1)-c*xEJM1) -
        bC*kC*xEJM1;
62 end
63
64 t = (0:n-1)*dT;
65
66 yL = {'\theta_x' '\theta_z' ...
67       '\omega_x' '\omega_z'};
68 TimeHistory(t,x,yL,'PulseResponse')
69 IL = {'Truth' 'Estimate'};
70 leg = {IL IL IL IL};
71 j = {[1 5] [2 6] [3 7] [4 8]};
72 TimeHistory(t,[x;xE],yL,'Estimator',j,leg);

```



Example 12.9: Low-bandwidth control simulation.

Table 12.1 Spacecraft parameters.

Parameter	Value
Mass	3 kg
I	$\begin{bmatrix} 0.025 & 0 & 0 \\ 0 & 0.025 & 0 \\ 0 & 0 & 0.005 \end{bmatrix}$ kg m ²
h_w	0.04 N m s
ω_0	0.0011 rad/s

The next step is to use magnetic torquers. For this example, we assume that we only want to use the torquers for roll and yaw. We have three degrees-of-freedom, but only two torques to match. The approach is to pick the three torquer commands so that the roll and yaw torques are met exactly and the third torque is minimized. The cost function is

$$J = (T - M^\times b)^T (TM^\times b) + M^T M \quad (12.79)$$

with two elements constrained to exactly match the desired torques. The algorithm is

$$s = b^\times \quad (12.80)$$

$$c = ss \quad (12.81)$$

$$\lambda = -\frac{T}{c(a, a)} \quad (12.82)$$

$$M = s(1 : 3, a)\lambda \quad (12.83)$$

where a is an index array with two elements.

If you wanted to also control pitch, this can only be done approximately using an algorithm such as

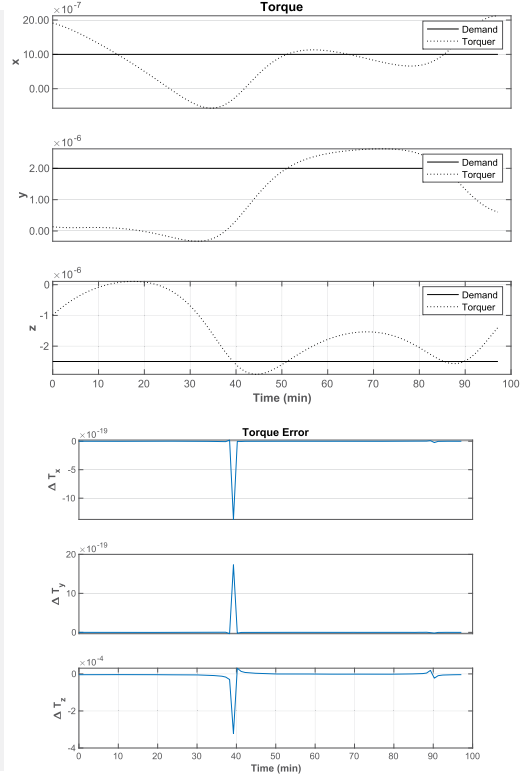
$$M = \frac{b^\times T}{|b|^2} \quad (12.84)$$

where b is the magnetic field and T is the desired torque, both in the body frame. The performance will be very dependent on the orbit. Example 12.10 shows the results for the two methods in a 28.6-deg orbit. The spike in torque error is due to the 2×2 matrix being ill-conditioned. This does not, at least in this orbit, pose a particular problem in computing the dipole.

```

1 a      = 7000;
2 e      = 0;
3 i      = 28.4*pi/180;
4 jD0    = Date2JD([4 4 2030]);
5
6 [r,v,t] = RVOrbGen([a i 0 0 e 0]);
7 jD      = jD0 + t/86400;
8
9 b      = BDipole(r,jD);
10 n      = length(t);
11
12 b      = QForm(QLVLH(r,v),b);
13
14 tD     = [1e-6;2e-6;-2.5e-6].*ones(1,n);
15 tM     = zeros(3,n);
16 m      = zeros(3,n);
17 for k = 1:n
18     m(:,k) = Skew(b(:,k))*tD(:,k)/Mag(b(:,k))^2;
19     tM(:,k) = Skew(m(:,k))*b(:,k);
20 end
21
22 IL = {'Demand' 'Torquer'} ;
23 leg = {IL IL IL};
24 j   = {[1 4] [2 5] [3 6]};
25 yL = {'x' 'y' 'z'};
26 TimeHistory(t,[tD;tM],yL,'Torque',j,leg);
27
28 % Pick x and y
29 a = [1;2];
30
31 for k = 1:n
32     s      = Skew(b(:,k));
33     c      = s*s;
34     cA     = c(a,a);
35     lambda = -cA\tD(a,k);
36     m(:,k) = s(:,a)*lambda;
37     tM(:,k) = Skew(m(:,k))*b(:,k);
38 end
39
40 yL = {'\Delta_T_x' '\Delta_T_y' '\Delta_T_z'};
41 TimeHistory(t,tD-tM,yL,'Torque_Error');

```



Example 12.10: Torque generation for magnetic torquers.

12.13. Lyapunov control

This section will use Lyapunov's direct method to control the nonlinear attitude dynamics and kinematics. The dynamical system is

$$T = I_c \dot{\omega} + \omega^\times (I_c \omega + I_w (\omega_w + \omega)) + J(\dot{\omega}_w + \dot{\omega}) \quad (12.85)$$

$$T_w = I_w (\dot{\omega}_w + \dot{\omega}) \quad (12.86)$$

$$q = f(q, \omega) \quad (12.87)$$

where q is the quaternion, ω is the angular rate of the core spacecraft, ω_w is the wheel angular rates with respect to the spacecraft core, T_w is the wheel torque, I_w is the wheel inertia, T is the external torque, and I_c is the inertia matrix for the core. The quaternion transforms from the inertial to the body frame. Three orthogonal wheels are used. This can be simplified if we let $I = I_c + I_w$.

$$T = I \dot{\omega} + \omega^\times I \omega + T_w + \omega^\times h_w \quad (12.88)$$

$$T_w = I_w(\dot{\omega}_w + \dot{\omega}) \quad (12.89)$$

$$q = f(q, \omega) \quad (12.90)$$

where $h_w = I_w \omega_w$. Lyapunov's direct method has two steps. The first is to find a scalar function of the states of the system, known as the Lyapunov function, and then to evaluate its derivative along the trajectory of the system. If the Lyapunov function is always decreasing the system will eventually reach equilibrium.

The controller will try to drive the body rates and the quaternion error to zero. Define the error quaternion as

$$q_E = q^T \otimes q_T \quad (12.91)$$

where q_T is the target quaternion. The error quaternion consists of a scalar and vector part,

$$q_E = \begin{bmatrix} s \\ u \end{bmatrix} \quad (12.92)$$

where u is a 3-by-1 vector and s is a scalar.

If there exists a Lyapunov function $V(x)$, where x is the state vector that is to be controlled, such that

1. $V(x)$ is positive-definite;
2. $\dot{V}(x)$ is negative-definite;
3. $V(x) \rightarrow \infty$ as $\|x\| \rightarrow \infty$;

then the equilibrium point is uniformly asymptotically stable. For this dynamical system [1]

$$V = \frac{1}{2} \omega^T I \omega + \eta ((1-s)^2 + u^T u) \quad (12.93)$$

The first time derivative is

$$\dot{V} = \frac{1}{2} (\omega^T I \dot{\omega} + \dot{\omega}^T I \omega) + \eta (2s\dot{s} + \dot{u}^T u + u^T \dot{u}) \quad (12.94)$$

or

$$\dot{V} = \omega^T I \dot{\omega} + \eta (2s\dot{s} + \dot{u}^T u + u^T \dot{u}) \quad (12.95)$$

After much algebra this becomes

$$\dot{V} = \omega^T (I \dot{\omega} + \eta u) \quad (12.96)$$

The control law is

$$T_w = \xi \omega + \eta u - \omega^\times h_W \quad (12.97)$$

Combining this with the dynamical equations results in

$$0 = I\dot{\omega} + \omega^\times I\omega + \xi\omega + \eta u \quad (12.98)$$

Substituting into $\dot{V}$

$$\dot{V} = \omega^T (-\omega^\times I\omega + \xi\omega) \quad (12.99)$$

Finally, since $\omega^T(\omega^\times I\omega) = 0$,

$$\dot{V} = -\omega^T \xi \omega \leq 0 \quad (12.100)$$

which is negative-semidefinite. This is a PD controller with compensation for the wheel-momentum torque.

The performance of the Lyapunov-maneuver control system is shown in Example 12.11.

12.14. Orbit-transfer maneuver

Hohmann transfers are a well-known orbital maneuver used to change the semimajor axis of an orbit.

In this demonstration, we create a 6U CubeSat that has 3 orthogonal reaction wheels and a single hydrazine thruster. The thruster is aligned with the x -axis and must be aligned with the velocity vector to do the maneuver. An ideal Hohmann maneuver is done with impulsive burns at two points in the orbit. In reality, with a thruster, we have to do a finite burn.

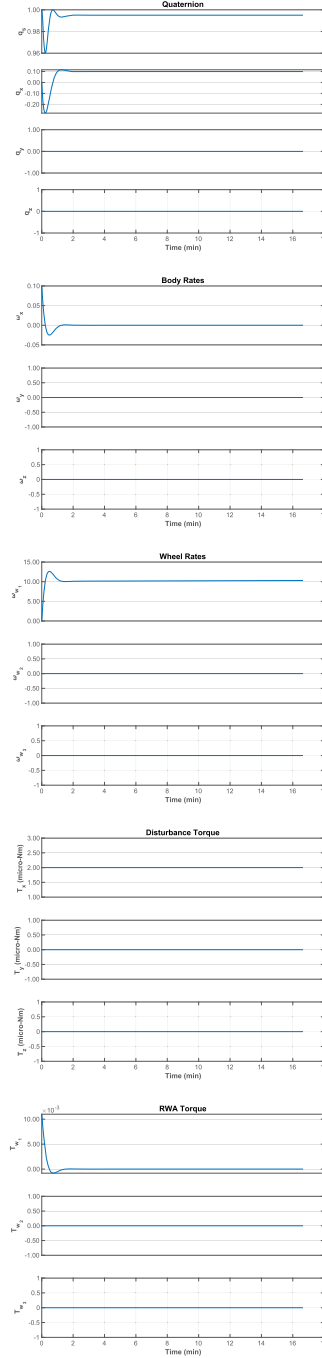
The simulation computes burn durations based on the thrust and the mass of the spacecraft. The maneuver is quite small, so the mass change is not important. The Hohmann transfer is computed with the following Spacecraft Control Toolbox code: The first time `OrbMnvrHohmann` is called it generates the first plot. This shows the initial orbit, the final orbit, and the transfer orbit. The attitude control system uses a proportional-integral-derivative control law. The entire simulation is in a short script. The spacecraft x -axis is aligned with the initial thrust vector so an attitude maneuver is not required. At the final orbit radius, an attitude maneuver is needed to reorient for the final burn.

The simulation is shown in Example 12.12. The spacecraft body rates, in the body frame, change only during the reorientation. The reaction-wheel rate changes are proportional to the body-rate changes. The wheel torques are small. The rocket engine is turned on during the “impulsive” burn times. The thruster is a 0.2-lbf hydrazine thruster that is based on the Aerojet-Rocketdyne MR-103. The semimajor axis and eccentricity are shown in the last plot. The middle portion is during the transfer orbit. The eccentricity is zero at the start and finish. Note the slope in both eccentricity and

```

1 %% Constants
2 controlOn = true;
3
4 %% User initialization
5 n = 1000; % Number of simulation steps
6 dT = 1.0; % Time step (s)
7 inertia = diag([1 2 2]);
8 qT = QUnit([1;0;1;0;0]);
9
10 % Control
11 xi = 0.1;
12 eta = 0.01;
13
14 % Initial Julian date
15 jD0 = Date2JD([2023 4 4]);
16
17 % Initial states
18 q = [1;0;0;0];
19 omega = [0;0;0;0]; % rad/s
20 omegaRWA = [0;0;0]; % rad/s
21 torqueD = [2;0;0]*1e-6; % Disturbance torque
22
23 %% State vector
24 x = [q;omega;omegaRWA];
25 dRHS = RHSGyrostat;
26 dRHS.inertia = inertia;
27
28 %% Simulate
29 xP = zeros(16,n);
30
31 % Simulation loop
32 for k = 1:n
33
34     if( controlOn )
35         q = x(1:4);
36         omega = x(5:7);
37         qE = QMult(QPose(q),qT);
38         tRWA = xi*omega + eta*qE(2:4) - cross(omega,
39             diag(dRHS.inrWheel)*x(8:10));
40     else
41         tRWA = [0;0;0];
42     end
43     dRHS.torque = torqueD;
44     dRHS.torqueWheel = tRWA;
45     xP(:,k) = [x;torqueD*1e6;tRWA];
46     x = RK4(@RHSGyrostat,x,dT,0,dRHS);
47 end
48
49 %% Plotting
50 t = (0:n-1)*dT;
51 yL = {'q_s' 'q_x' 'q_y' 'q_z' ...
52     '\omega_x' '\omega_y' '\omega_z' ...
53     '\omega_{w_1}' '\omega_{w_2}' '\omega_{w_3}' ...
54     'T_{x_\square}(\text{micro-Nm})' 'T_{y_\square}(\text{micro-Nm})' 'T_{z_\square}(\text{micro-Nm})' ...
55     'T_{w_1}' 'T_{w_2}' 'T_{w_3}'};
56
57 tL = {'Quaternion' 'BodyRates' 'WheelRates' ...
58     'DisturbanceTorque' 'RWA_Torque'};
59
60 kL = [1:4 5:7 8:10 11:13 14:16];
61 for j = 1:length(tL)
62     k = kL{j};
63     TimeHistory(t,xP(k,:),yL(k),tL{j});
64 end
65
66 Fig1

```

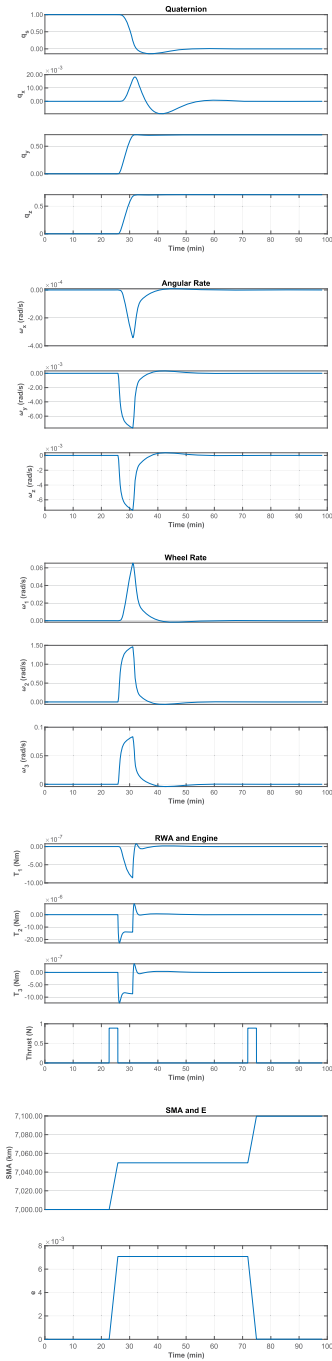


Example 12.11: Lyapunov-maneuver control.

```

1 nToKN = 0.001;
2 mu = Constant('mu_earth');
3 % Compute the maneuver
4 rI = [-7000;0;0];
5 vI = [0;-sqrt(mu/Mag(rI));0];
6 rF = 7100;
7 eI = RV2El(rI,vI);
8 p = Period(eI(1));
9 OrbMnvrHohmann(Mag(rI),rF);
10 [dV,tOF] = OrbMnvrHohmann(Mag(rI),rF);
11 % Set up the spacecraft
12 mP = 6; % kg
13 thrustE = 4.448e0.2; % N
14 dVTot = dV.a + dV.b; % km/s
15 iSp = 224; % s
16 fS = 0.1; % Structural fraction
17 [mF,mT] = RocketMass(iSp,mP,fS,dVTot);
18 inr = InertiaCubeSat('6U',mT);
19 acc = thrustE/mT; % m/s^2
20 tBA = dV.a/acc/nToKN; % s
21 tBB = dV.b/acc/nToKN; % s
22 dRHS = RRSRWAOrbit;
23 dRHS.inr = inr;
24 dRHS.m = nT;
25 tStart = [p/4-tBA/2 p/4+tOF-tBB/2];
26 tEnd = tStart + [tBA tBB];
27
28 % Set up the controller
29 dC = PID3Axis;
30 dC.body_vector = [1;0;0];
31 dC.mode = 1; % Align two axes
32 dC.inertia = inr;
33
34 fprintf('Burn_A_duration,%8.2f\sn',tBA);
35 fprintf('Burn_B_duration,%8.2f\sn',tBB);
36 fprintf('Thrust,,,,,,,,,%8.2f\Nn',thrustE);
37 fprintf('Mass_Total,,,,,,,,,%8.2f\Nn',mT);
38 fprintf('Mass_Fuel,,,,,,,,,%8.2f\kg\Nn',mF);
39 fprintf('Initial_SMA,,,,,,,,,%8.2f\km\Nn',eI(1));
40 fprintf('Initial_e,,,,,,,,,%8.2f\Nn',eI(5));
41
42 % Simulation
43 % ECI burn vector
44 uBurn = [1 -1;0 0;0];
45 dT = 1; % s
46 n = ceil(2*tOF/dT);
47 kMnvr = 1;
48 x = [rI;vI;1;zeros(9,1)];
49 xP = zeros(22,n);
50 inMnvr = false;
51 t = (0:n-1)*dT;
52 % Simulation loop
53 for k = 1:n
54     dC.eci_vector = uBurn(:,kMnvr);
55     [RWA,dC] = PID3Axis(x(7:10),dC);
56     inMnvr = false;
57     if( t(k) > tStart(1) && t(k) < tEnd(1) )
58         inMnvr = true;
59     end
60     if( t(k) > tEnd(1) )
61         kMnvr = 2;
62     end
63     if( t(k) > tStart(2) && t(k) < tEnd(2) )
64         inMnvr = true;
65     end
66     if( inMnvr )
67         dRHS.force = thrustE*QTForm(x(7:10),dC,
68             body_vector)/nToKN; % kN
69     else
70         dRHS.force = [0;0;0];
71     end
72     eI = RV2El(x(1:3),x(4:6));
73     xP(:,k) = [x;RWA;Mag(dRHS.force)/nToKN;eI(1);
74         eI(5)];
75     dRHS.torqueRWA = -(RWA;
76     x = RK4(@RRSRWAOrbit,x,dT,0,dRHS);
77 end
78 fprintf('Final_SMA,,,,,,,,,%8.2f\km\Nn',eI(1));
79 fprintf('Final_e,,,,,,,,,%8.2f\Nn',eI(5));
80 yL = {'r_x_0(km)' 'r_y_0(km)' 'r_z_0(km)' ...
81     'v_x_0(km/s)' 'v_y_0(km/s)' 'v_z_0(km/s)' ...
82     'q_x' 'q_y' 'q_z' 'q_4' ...
83     '\omega_x' '\omega_y' '\omega_z' ...
84     '\omega_1' '\omega_2' '\omega_3' ...
85     'T_1_0(Nm)' 'T_2_0(Nm)' 'T_3_0(Nm)' ...
86     'Thrust(N)' 'SMA_(km)' 'e'};
87 k=1:3;TimeHistory(t,xP(k,:),yL(k),'Position');
88 k=4:6;TimeHistory(t,xP(k,:),yL(k),'Velocity');
89 k=7:10;TimeHistory(t,xP(k,:),yL(k),'Quatation');
90 k=11:13;TimeHistory(t,xP(k,:),yL(k),'Body_Rate');
91 k=14:16;TimeHistory(t,xP(k,:),yL(k),'RWA_Rate');
92 k=17:20;TimeHistory(t,xP(k,:),yL(k),'Actuators');
93 k=21:22;TimeHistory(t,xP(k,:),yL(k),'SMA_and_E');
94 ;
95
96 Figui;

```



Example 12.12: Attitude maneuvers for an orbit transfer.

semimajor axis due to the finite acceleration. The maneuver is not perfect because the burn durations are finite.

The simulation also prints out some results

Hohmann Transfer

Initial Orbit Radius	=	7000.0000
Final Orbit Radius	=	7100.0000
Delta V Total	=	0.0533
Delta V at A	=	0.0267
Delta V at B	=	0.0266
E transfer	=	0.0071
SMA transfer	=	7050.0000
Time of Flight	=	0.8182 hours
Burn A duration	185.04 s	
Burn B duration	184.39 s	
Thrust	0.89 N	
Mass Total	6.16 kg	
Mass Fuel	0.15 kg	
Initial SMA	7000.00 km	
Initial e	0.00	
Final SMA	7099.72 km	
Final e	0.00	

12.15. Docking

Docking involves aligning the spacecraft docking adaptor with the target docking point and then controlling the spacecraft so that it docks with zero relative velocity, both angular and linear, zero angular error, and zero position error. It is a six-degrees-of-freedom problem.

For this example, we will use a Soyuz spacecraft shown, with thruster locations, in Fig. 12.10. Attitude and position will be controlled simultaneously. The dynamical orbital equations are Hill's equations [2]. These are relative orbit equations linearized about a nominal circular orbit. The Hill's frame is shown in Fig. 12.9.

The dynamical equations are linear and in state-space form are:

$$\dot{\mathbf{z}} = \mathbf{a}\mathbf{z} + \mathbf{b}\mathbf{u} \quad (12.101)$$

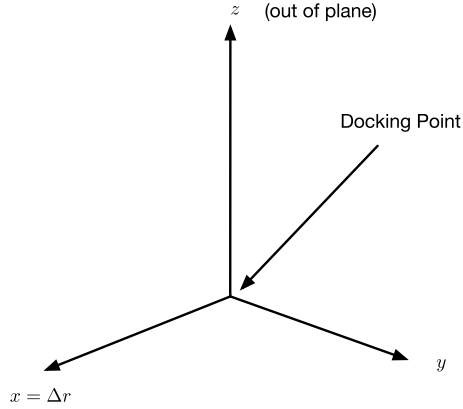


Figure 12.9 Hill's frame of reference for relative orbital motion.

where the state-transition matrix is

$$a = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 3n^2 & 0 & 0 & 0 & 2n & 0 \\ 0 & 0 & 0 & -2n & 0 & 0 \\ 0 & 0 & -n^2 & 0 & 0 & 0 \end{bmatrix} \quad (12.102)$$

where n is orbit rate. The input matrix, which multiplies the control accelerations, is

$$b = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (12.103)$$

The state vector is

$$s = \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix} \quad (12.104)$$

where x and y are inplane. z is out-of-plane. Its dynamical equation for z is that of an undamped oscillator. x and y are coupled. y does not appear explicitly in the equations and can be large. The state equations are used to produce a full-state feedback-control law if we define the weighted control as

$$C = \frac{1}{2} (s^T Q s + u^T R u) \quad (12.105)$$

where Q is the state cost and R is the control cost. The solution, P , to the matrix Riccati equation

$$A^T P + P A - P B R^{-1} B^T P - Q = 0 \quad (12.106)$$

and the controller is

$$u = -R^{-1} B^T P s \quad (12.107)$$

or

$$u = -k s \quad (12.108)$$

where

$$k = R^{-1} B^T P \quad (12.109)$$

The force is mu , where m is the spacecraft mass. This assumes that the full state s can be measured. We also assume that the docking port is at the origin. The state measurement is done in a variety of ways. Many were demonstrated on the Swedish Space Corporation's Prisma mission [3]. Spacecraft docking with the International Space Station have multiple sensors. The sensor suite varies from spacecraft to spacecraft. Differential GPS, radar, and optical sensors are all used.

Linear programming is used to distribute forces and torques among the thrusters. Linear programming solves the problem

$$\begin{bmatrix} F \\ T \end{bmatrix} = A u \quad (12.110)$$

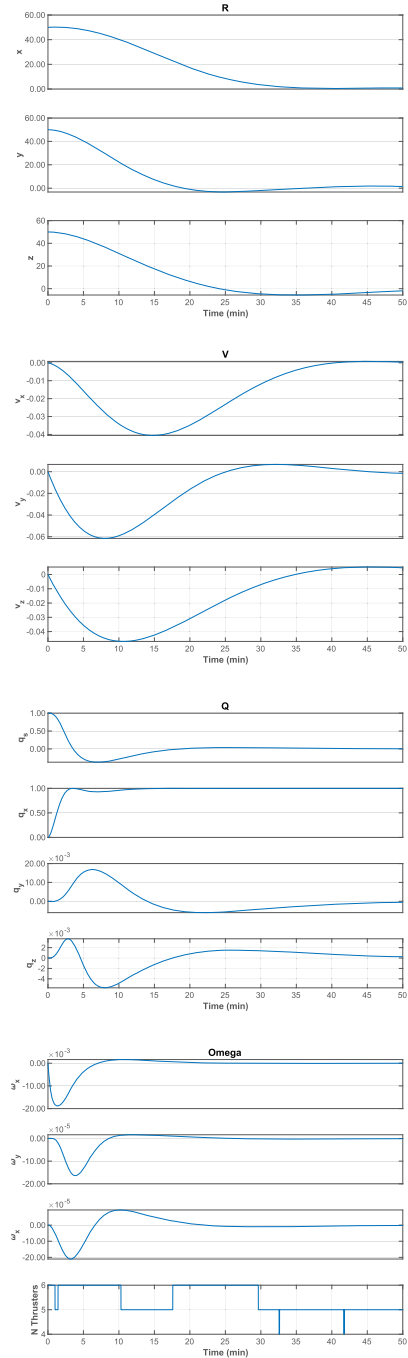
where u is the thruster throttle setting and A is a 6-by- n matrix of force and torque vectors, where n is the number of thrusters. The throttle setting, u , can be converted to a pulsewidth. This allows the spacecraft to easily accommodate failed thrusters. It is also easy to not use thrusters that fire towards the target spacecraft. Fig. 12.10 shows the thruster layout on a Soyuz spacecraft.

The attitude controller is a PID three-axis controller. The thrusters are pulsewidth modulated and provide translation and attitude control simultaneously. The minimum number of thrusters needed for six-degrees-of-freedom control is seven. More may be needed if saturation is considered. Example 12.13 shows the operation of the controller. In practice, the alignment operation could be done first, and then the docking.

```

1 syz = load('Soyuz.mat');
2 dT = 0.5;
3 nSim = 6000;
4 sMA = 7000;
5 % Dynamics
6 n = 2*pi/Period(sMA);
7 [a, b] = LinOrb([], n);
8 dRHS = syz; % Soyuz thruster data
9 dRHS.a = a; % Linearized dynamics
10 dRHS.b = b;
11 % Full state feedback gain for the orbit
12 q = 1e-15*eye(6);
13 r = 1e-4*eye(3);
14 kO = QCR(a, b, q, r);
15 eig(a-b*kO)
16 % PID controller for attitude
17 dC = PID3Axis;
18 [dC.a, dC.b, dC.c, dC.d] = PIDMIMO
    (1, 1, 0.004, 500, 0.04, dT);
19 dC.q_desired_state = [0; 1; 0; 0];
20 % Simulate
21 xP = zeros(14, nSim);
22 x = [50; 50; 50; 0; 0; 1; zeros(6, 1)];
23 A = [dRHS.aTorque; dRHS.aForce];
24 err = zeros(1, nSim);
25 options = optimoptions('linprog', 'Display', 'none'
    );
26 uMax = zeros(16, 1);
27 for k = 1:nSim
28     [acc, dC] = PID3Axis(x(7:10), dC);
29     torqueC = dRHS.inertia*acc;
30     forceC = QForm(x(7:10), -kO*x(1:6)*dRHS.mass);
31     b = [torqueC; forceC];
32     dRHS.u = linprog(ones(1, 16), [], [], A, b, zeros
        (1, 16), [], [], options);
33     err(k) = Mag(b - A*dRHS.u);
34     xP(:, k) = [x; length(find(dRHS.u > 1e-8))];
35     x = RK4(@RHS, x, dT, 0, dRHS);
36     uMax = max([uMax abs(dRHS.u)], [], 2);
37 end
38 close %% Plotting
39 yL = {'x' 'y' 'z' 'v_x' 'v_y' 'v_z' ...
40     'q_s' 'q_x' 'q_y' 'q_z' '\omega_x' ...
41     '\omega_y' '\omega_z' 'N_Thrusters'};
42 t = (0:nSim-1)*dT; k = 1:3;
43 TimeHistory(t, xP(k,:), yL(k), 'R'); k = 4:6;
44 TimeHistory(t, xP(k,:), yL(k), 'V'); k = 7:10;
45 TimeHistory(t, xP(k,:), yL(k), 'Q'); k = 11:14;
46 TimeHistory(t, xP(k,:), yL(k), 'Omega');
47
48 %% Dynamical equations
49 function xDot = RHS(x, ~, d)
50     force = d.aForce*d.u;
51     torque = d.aTorque*d.u;
52     accel = QTForm(x(7:10), force)/d.mass;
53     q = x(7:10); omega = x(11:13);
54     omegaDot = d.inertia \ (torque ...
55         - Cross(omega, d.inertia*omega));
56     qDot = QTToBDot(q, omega);
57     xDot = [d.a*x(1:6)+d.b*accel; qDot; omegaDot];
58 end

```



Example 12.13: Docking simulation.

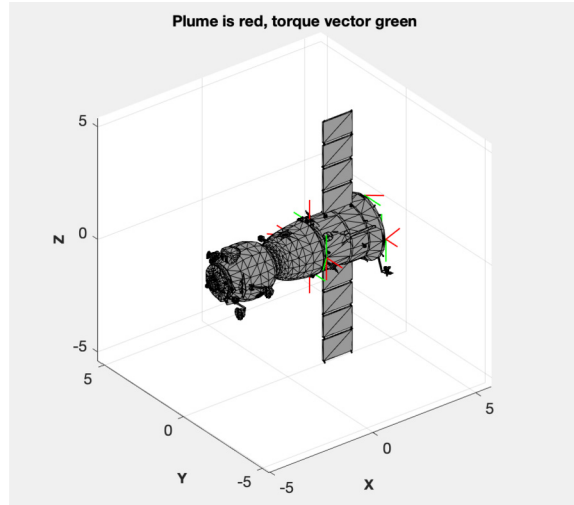


Figure 12.10 The Soyuz spacecraft with thruster locations.

12.16. Command distribution

Often, a 3-axis torque command, or a 6-axis force and torque command must be distributed to multiple actuators. One approach is to design a multioutput control algorithm with one output for each actuator. This becomes complicated when fault-tolerance and redundancy management require that the system operates with subsets of the actuators or when the number of actuators is very large (the Shuttle Orbiter has 44 thrusters). A more practical approach is to decouple the torque and force distribution from the control.

When distributing torque in this manner, it is important that the actuators have similar dynamics, and that these dynamics are accounted for in the control design. One must be careful when trying to treat a DC motor like a pulsewidth-modulated thruster!

The following sections discuss the use of techniques to distribute torque to a set of actuators. The final section discusses logic approaches.

12.16.1 The optimal torque-distribution problem

Given n actuators, all with similar dynamics as discussed above, three equality constraints, and m inequality constraints on the magnitude of the actuation, the general cost function to be minimized can be written as

$$L = C(u) + \lambda^T (T - Bu) + \mu^T u \quad (12.111)$$

where u is the control vector of order n , $C(u)$ is the scalar cost function to be applied to the control, and λ is the 3-vector of Lagrange multipliers used to adjoin the equality

constraints, T is the desired torque vector, B is the torque distribution matrix, and μ is the vector of Lagrange multipliers used to adjoin inequality constraints on the control.

Inequality constraints come in a number of forms. For a scalar actuator, one in which the torque is the product of a scalar controlled by the controller and a fixed unit vector, the effective scalar actuation is

$$u_{\text{eff}} = \begin{cases} f(u) & u_{\min} \leq u \leq u_{\max} \\ 0 & \text{otherwise} \end{cases} \quad (12.112)$$

This would apply to a fixed-orientation thruster (such as the Shuttle Orbiters Reaction Control System (RCS) thrusters, but not its Orbital Maneuvering System (OMS) thrusters) a reaction wheel, a magnetic torquer, a control-moment gyro (CMG) gimbal actuator, etc. Generally, the effective output is a nonlinear function of the input (throttle setting, voltage, pulse duration, etc.). Unfortunately, trying to minimize (12.111) with a function like (12.112) leads us to nonlinear optimization that, at present, is impractical in a real-time system.

In many cases, the actuator can be modeled as

$$u_{\text{eff}} = \begin{cases} u & u \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad (12.113)$$

for a unidirectional actuator and

$$u_{\text{eff}} = u \quad (12.114)$$

for a bidirectional actuator such as a reaction wheel. If (12.114) is used, $\mu = 0$ and drops out of the cost function. If (12.113) is used, we still have the simple constraint on u .

The next step is to select an appropriate cost functional. For fixed-orientation thrusters, fuel is to be minimized, which means

$$C = \sum_{i=1}^n c_i u_i \quad (12.115)$$

For a motor energy is usually minimized

$$C = \sum_{i=1}^n c_i u_i^2 \quad (12.116)$$

If Eq. (12.116) is the cost functional, and (12.114) is the control, the optimal command vector can be found in a straightforward manner. Combine (12.111), (12.114), and (12.116), where (12.116) has been replaced by the equivalent vector notation, and c is a diagonal matrix whose components are the individual costs

$$L = \frac{1}{2} u^T c u + \lambda^T (T - Bu) \quad (12.117)$$

The partial derivative of L with respect to u is the vector equation

$$\frac{\partial L}{\partial u} = cu - B^T \lambda \quad (12.118)$$

Taking the derivative of (12.117) with respect to λ gives the equality constraint $T - Bu = 0$. Substituting (12.118) into the equality constraint gives the equation

$$T = Bc^{-1}B^T \lambda \quad (12.119)$$

Solving for λ and replacing λ in (12.118) with the result gives

$$u = c^{-1}B^T(Bc^{-1}B^T)^{-1}T \quad (12.120)$$

which is the familiar formula for the pseudoinverse. This solution minimizes the square of the command, so it is in some sense a minimum-energy solution. It also penalizes large values of u most strongly, so it will tend to keep the actuators away from their limits. Since c and B are constant, the pseudoinverse need not be computed onboard.

12.16.2 Reaction wheels

Many Earth-pointing spacecraft use four reaction wheels in a pyramid configuration about one of the axes. That axis is usually the one about which the spacecraft rotates as it goes around its orbit. Assume that four reaction wheels are in a pyramid and each is canted θ deg from the z -axis. The control matrix is 3×4 and in this case is

$$B = \begin{bmatrix} \sin \theta & -\sin \theta & 0 & 0 \\ 0 & 0 & \sin \theta & -\sin \theta \\ \cos \theta & \cos \theta & \cos \theta & \cos \theta \end{bmatrix} \quad (12.121)$$

If all four wheels have equal costs, c is the identity matrix and the torque distribution matrix is the 4×3 matrix

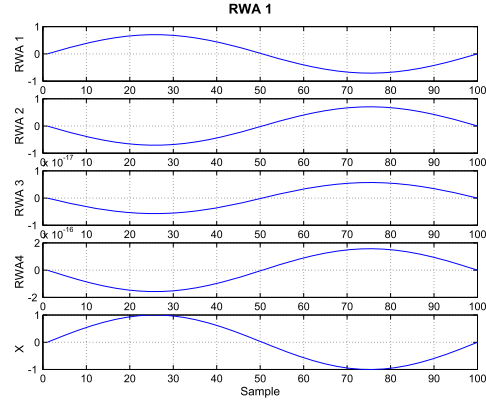
$$B^T(BB^T)^{-1} \quad (12.122)$$

All four wheels will contribute equally to any z -axis torque demand. Wheel one will respond to a positive x -axis torque demand with a positive command and wheel 2 will respond with a negative command, (12.122), which is known as the pseudoinverse function since it is the least-squares approximation to an inverse for a nonsquare matrix. Example 12.14 varies the x -axis torque sinusoidally. Note that the 1 and 2 wheels produce equal and opposite torques.

```

1 theta = pi/4; n = 100;
2 c = cos(theta); s = sin(theta);
3 b = [s -s 0 0; 0 0 s -s; c c c c];
4 p = pinv(b);
5 t = [sin(linspace(0,2*pi,n)); zeros(2,n)];
6 for k = 1:n
7     tRWA(:,k) = p*t(:,k);
8 end
9 h = Plot2D(1:n,[tRWA;t(1,:)],'Sample',{ 'RWA_1' '
    RWA_2' 'RWA_3' 'RWA4', 'X' })
10 set(h,'color',[1 1 1]);
11 PrintFig(0,1,1,'RWA Torque')

```



Example 12.14: Reaction-wheel pyramid.

12.16.3 Linear programming

Given the problem

$$x = Au \quad (12.123)$$

$$\min(c^T u) \quad (12.124)$$

$$u \geq 0 \quad (12.125)$$

where the length of u is greater than the length of x , the solution can be found using the simplex algorithm. It is also possible to handle a magnitude constraint on u without adding additional constraint equations. Simplex is ideally suited for thrusters that are unidirectional. We could use the pseudoinverse approach if we had matched pairs of thrusters.

Given that x is length n and u is length m , simplex finds the subset of length n of u ($m > n$) such that

$$u = A(k, k)^{-1}x \quad (12.126)$$

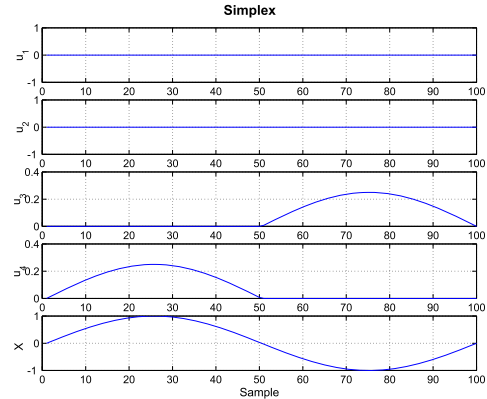
where k is a subset of u of length n . Without a magnitude constraint, the number of actuators that are nonzero exactly equals the number of constraint equations. With a magnitude constraint, additional actuators will be used.

Example 12.15 is to set up North-South station-keeping control using simplex. Since we can only control two axes of torques our torque matrix is 2-by- n , not 3-by- n . Simplex picks the correct thruster and scales the thrust. Thruster scaling would have to be implemented using pulsewidth modulation.

```

1 n = 100;
2 r = [1 -1 0 0; 0 0 1 -1; 1 1 1 1];
3 v = [0 0 0 0; 0 0 0 0; -1 -1 -1 -1];
4 a = 4*Cross(r,v);
5 t = [sin(linspace(0,2*pi,n)); zeros(1,n)];
6 for k = 1:n
7     [u(:,k), f] = Simplex(ones(1,4),a(1:2,:),t(1:2,
8         k),1);
9 end
10 h = Plot2D(1:n,[u;t(1,:)],'Sample',...
11     ['u_1' 'u_2' 'u_3' 'u_4', 'X'],...
12     'Simplex')
13 set(h,'color',[1 1 1]);
14 PrintFig(0,1,1,'Simplex')

```



Example 12.15: Linear programming solution for north-face thruster torque distribution.

12.17. Attitude profile design

The attitude profile of a spacecraft mission is the planned sequence of spacecraft orientations or “pointing modes” over time. The design of an attitude profile can be challenging, as it typically requires different parts of the spacecraft to simultaneously point in different directions.

For example, suppose we have a sensor to point down towards the Earth and solar panels that should point towards the Sun. The sensor is fixed to the spacecraft bus, and the solar panels may be rotated about a common axis. The attitude pointing objective, in this case, is to point the sensor boresight axis in the nadir direction and point the solar panel axis normal to the Sun vector. In this way, the solar panels may rotate about the axis to point their surface towards the Sun. Given the relative locations of the Earth and Sun with respect to the satellite, it may not be possible to completely satisfy both pointing objectives. A variety of methods may be used to determine a single orientation that provides an acceptable solution.

This section discusses one possible method for defining the attitude pointing mode as a combination of primary and secondary alignments. The primary alignment aligns a given body vector with a given target vector defined in the inertial frame. The definition of the target inertial vector depends upon the alignment type. Some of the possible alignment types include:

- Nadir pointing;
- Sun pointing;
- Orbit normal;
- Latitude-longitude pointing;
- Local vertical/local horizontal (LVLH) pointing.

The secondary alignment is achieved by rotating the spacecraft about the primary body vector, maintaining the primary alignment until the secondary body vector is as closely aligned as possible to the secondary target. The secondary target may be derived from the same menu of alignment types.

12.17.1 Alignment method

For both the primary and secondary alignments, we choose a body vector on the spacecraft to be aligned with an inertial vector. Let $\vec{b}_1$ and $\vec{b}_2$ be the body vectors for the primary and secondary alignments, respectively. Similarly, let $\vec{u}_1$ and $\vec{u}_2$ be the target inertial vectors.

The desired inertial to body quaternion for the primary alignment is found by computing the quaternion that rotates $\vec{u}_1$ to $\vec{b}_1$. This may be done with the SCT function U2Q.m.

```
>> q_1 = U2Q( u1, b1 );
```

This quaternion is defined as:

$$q_1 = \begin{bmatrix} s/2 \\ d_x/s \\ d_y/s \\ d_z/s \end{bmatrix}$$

where

$$\vec{d} = \frac{\vec{u}_1 \times \vec{b}_1}{|\vec{u}_1||\vec{b}_1|}$$

and

$$s = \sqrt{2 \left(1 + \frac{\vec{u}_1 \cdot \vec{b}_1}{|\vec{u}_1||\vec{b}_1|} \right)}$$

The secondary alignment is achieved by rotating about vector $\vec{b}_1$. Let this rotation be denoted by quaternion q_{Rot} . The full ECI-to-body quaternion is therefore:

$$q_{EB} = q_1 q_{Rot}$$

where standard quaternion multiplication is used. The derivation of q_{Rot} is provided in the next section.

12.17.2 Minimizing the separation angle between vectors

Consider three unit vectors, $\vec{a}$, $\vec{b}$, $\vec{c}$, all defined in the body frame of the spacecraft. From the previous section, define $\vec{a} = \vec{b}_1$ to be the axis about which we may rotate. Define

$\vec{b} = \vec{b}_2$ as the body-fixed vector that we wish to align with vector $\vec{c}$. Finally, define $\vec{c}$ as the secondary inertial target in the body frame by rotating $\vec{u}_2$ with q_{EB} . We wish to rotate the $\vec{b}$ vector about axis $\vec{a}$ so that $\vec{b}$ is as closely aligned with $\vec{c}$ as possible.

To closely align $\vec{b}$ with $\vec{c}$, we need to minimize the separation angle between the two vectors. The separation angle α is found from the dot product definition:

$$\cos \alpha = \vec{b}^* \cdot \vec{c}^* \quad (12.127)$$

where $\vec{b}^*$ is the new value of $\vec{b}$ after rotation. Both vectors are expressed in the xyz coordinate system. The geometry is illustrated in Fig. 12.11. A new coordinate system xy^*z is defined with x along $\vec{a}$, $z = \vec{a} \times \vec{b}$, and y completes the right-hand system with $y = z \times x$. The $\vec{b}$ vector lies in the xy -plane. This plane is rotated through θ to place $\vec{b}^*$ in the same plane with $\vec{a}$ and $\vec{c}$.

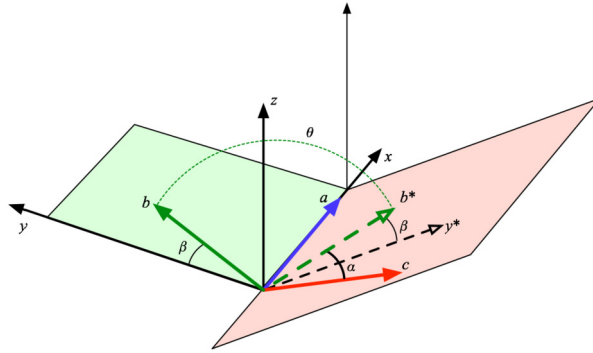


Figure 12.11 Rotation about an axis to align two vectors.

The desired vector $\vec{b}^*$ expressed in the xyz frame is:

$$\vec{b}^* = \begin{bmatrix} x^T \vec{b} \\ y^T \vec{b} \cos \theta \\ z^T \vec{b} \sin \theta \end{bmatrix} \quad (12.128)$$

The target vector $\vec{c}^*$ expressed in the xyz frame is:

$$\vec{c}^* = \begin{bmatrix} x^T \vec{c} \\ y^T \vec{c} \\ z^T \vec{c} \end{bmatrix} \quad (12.129)$$

Combining Eq. (12.128) and Eq. (12.129) into Eq. (12.127), we have:

$$\cos \alpha = (x^T \vec{b})(x^T \vec{c}) + (y^T \vec{b})(y^T \vec{c}) \cos \theta + (y^T \vec{b})(z^T \vec{c}) \sin \theta \quad (12.130)$$

To minimize α , we seek to maximize $\cos \alpha$. The maximum occurs when the derivative of $\cos \alpha$ with respect to θ becomes zero.

$$\frac{d(\cos \alpha)}{d\theta} = \gamma^T \vec{b} (-\sin \theta \gamma^T \vec{c} + \cos \theta z^T \vec{c}) = 0 \quad (12.131)$$

This leads to the simple solution:

$$\tan \theta = \frac{z^T \vec{c}}{\gamma^T \vec{c}} \quad (12.132)$$

With the axis and angle of rotation now defined, the quaternion to perform this rotation is:

$$q_{Rot} = \begin{bmatrix} \cos(\theta/2) \\ -a_x \sin(\theta/2) \\ -a_y \sin(\theta/2) \\ -a_z \sin(\theta/2) \end{bmatrix}$$

These equations result in the following algorithm:

```
x = Unit(a);
z = Unit(Cross(x,b));
y = Unit(Cross(z,x));
m = [x';y';z'];
rotAngle = atan2(z'*ut,y'*ut);
xc = x'*b;
yc = y'*b;
ub = m*[xc*ones(size(rotAngle)); yc*cos(rotAngle); yc*sin(rotAngle)];
c = ub(1,:).*ut(1,:)+ub(2,:).*ut(2,:)+ub(3,:).*ut(3,:);
k = find(abs(c)>1);
c(k)=sign(c(k)); % do this to prevent rounding errors from causing |c|>1
sepAngle = acos(c);
qRot = [cos(rotAngle/2);...
        sin(rotAngle/2)*axis(1);...
        sin(rotAngle/2)*axis(2);...
        sin(rotAngle/2)*axis(3)];
```

12.17.3 Computing the target-inertial vector

Let the target-inertial vector be $\vec{u}_T$. The definition of $\vec{u}_T$ is provided below for several common alignment modes. For convenient notation, let $[\]_u$ represent the unit operator on a vector, returning a vector of the same direction with length 1.

12.17.3.1 Sun pointing

$$\vec{u}_T = [\vec{s} - \vec{r}]_u$$

where $\vec{r}$ is the spacecraft position in the ECI frame, and $\vec{s}$ is the position of the Sun in the ECI frame. Note that $\vec{s}$ may be found from the SCT function SunV1.m or SunV2.m by supplying the Julian date epoch.

12.17.3.2 Nadir pointing

$$\vec{u}_T = -[\vec{r}]_u$$

12.17.3.3 Latitude–longitude pointing

$$\vec{u}_T = [M^T \vec{g}_{EF} - \vec{r}]_u$$

where M is the matrix that rotates from ECI to the Earth-fixed frame, and $\vec{g}_{EF}$ is the Earth-fixed coordinates for a ground site. Given a latitude λ and longitude ℓ , the ground-site vector $\vec{g}_{EF}$ is:

$$\vec{g}_{EF} = R \begin{bmatrix} \cos \lambda \cos \ell \\ \cos \lambda \sin \ell \\ \sin \lambda \end{bmatrix}$$

Note that M may be found using the SCT function TruEarth.m.

12.17.3.4 Orbit-normal pointing

$$\vec{u}_T = [\vec{r} \times \vec{v}]_u$$

where $\vec{v}$ is the ECI velocity of the spacecraft.

12.17.3.5 LVLH pointing

$$\vec{u}_T = M^T \vec{v}$$

where M rotates from the ECI frame to a local vertical/local horizontal (LVLH) frame, and $\vec{v}$ is a unit vector defined in the LVLH frame. The rotation matrix M may be computed from the spacecraft ECI position and velocity vectors as follows:

$$M = \begin{bmatrix} x^T \\ y^T \\ z^T \end{bmatrix}$$

where $y = [\vec{v} \times \vec{r}]_u$, $z = -[r]_u$, and $x = y \times z$.

12.18. Actuator sizing

The actuators must be sized to have sufficient torque to do required attitude maneuvers and to compensate for external disturbances. Moment-exchange devices, such as reaction wheels and control-moment gyros (CMG), must have sufficient momentum-storage capability for required maneuvers and to store momentum in between momentum-dumping maneuvers if using thrusters. If you are using magnetic torquers for unloading that ARE operating all the time then the momentum-storage capacity is based on whatever the magnetic torquers cannot instantaneously remove. Sizing of magnetic-torquer systems is complicated by the changes in the planet's magnetic field as the spacecraft moves in its orbit.

12.18.1 Maneuvers

The simplest maneuver is a rotation about an axis using a bang-bang maneuver. This means the actuator is either full on or full off. Assume the desired maneuver angle is θ

$$\frac{\theta}{2} = \frac{1}{2} \frac{T}{I} \left(\frac{\tau}{2} \right)^2 \quad (12.133)$$

where T is the torque, I the inertia about that axis, and τ is the maneuver duration. The required torque is

$$T = \frac{4\theta I}{\tau^2} \quad (12.134)$$

If we are using a momentum-exchange device, the required momentum storage is

$$h_w = \frac{\tau}{2} T = \frac{2\theta I}{\tau} \quad (12.135)$$

If a momentum or reaction wheel is used the wheel speed will be

$$\omega_w = \frac{h_w}{I_w} \quad (12.136)$$

where I_w is the wheel inertia. If a CMG is used the CMG angle is

$$\theta = \sin^{-1} \left(\frac{h_w}{h_{CMG}} \right) \quad (12.137)$$

where h_{CMG} is the momentum stored in the CMG.

The result will be slightly different for different controllers. Assume a PD controller with $\zeta = 1$. The second-order equation is

$$I(\ddot{\theta} + 2\omega_n\dot{\theta} + \omega_n^2\theta) = T \quad (12.138)$$

where ω_n is the natural frequency of the controller. The forward gain is $I\omega_n^2$. The solution is from the characteristic equation

$$s^2 + 2\omega_n s + \omega_n^2 = 0 \quad (12.139)$$

The solution for repeated roots is

$$\theta(t) = c_1 e^{-\omega_n t} + c_2 t e^{-\omega_n t} \quad (12.140)$$

If $\theta(0) = \theta_0$ and $\dot{\theta}(0) = 0$. The coefficients are

$$c_1 = \theta_0 \quad (12.141)$$

$$c_2 = \omega_n \theta_0 \quad (12.142)$$

$$\theta = \theta_0 e^{-\omega_n t} (1 + \omega_n t) \quad (12.143)$$

The control torque is

$$T = -I(2\omega_n \dot{\theta} + \omega_n^2 \theta) \quad (12.144)$$

or

$$T = \omega_n^2 I \theta_0 e^{-\omega_n t} (1 - \omega_n t) \quad (12.145)$$

The peak torque is

$$T = \omega_n^2 I \theta_0 \quad (12.146)$$

This can be compared with a bang-bang maneuver by selecting an acceptable final angle, θ_f , and computing ω_n numerically from the equation

$$\frac{\theta_f}{\theta_0} = e^{-\omega_n \tau} (1 + \omega_n \tau) \quad (12.147)$$

Example 12.16 compares a bang-bang maneuver with a PD maneuver. The PD requires much larger torque at the start. The PD controller reaches a higher angular velocity that will result in more momentum stored in the momentum-exchange device if momentum-exchange devices are used. The peak torque is also higher than with the bang-bang maneuver.

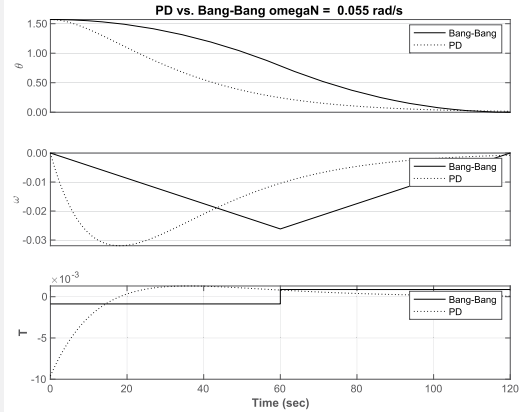
12.18.2 Disturbances

Estimating the size of momentum-exchange devices for disturbance accommodation can be done using a Fourier-series representation of the disturbance torque. For exam-

```

1 %% PD vs bang bang maneuver
2
3 inr      = 2;
4 theta0   = pi/2;
5 tau      = 120;
6 torqueBBMax = 4*theta0*inr/tau^2;
7
8 thetaF    = 0.01*theta0;
9 omegaN    = FindOmegaN(thetaF/theta0, tau);
10 torquePDMax = (theta0*inr)*omegaN^2;
11
12 n        = 100000;
13 dT       = tau/n;
14 xP1      = zeros(3,n+1);
15 xP2      = zeros(3,n+1);
16 x        = [theta0;0];
17 torque    = -torqueBBMax;
18 for k = 1:n
19     xP1(:,k) = [x;torque];
20     if( k < n/2)
21         torque = -torqueBBMax;
22     else
23         torque = torqueBBMax;
24     end
25     x = RK4(@RHS,x,dT,0,torque,inr);
26 end
27 xP1(:,k+1) = [x;torque];
28
29 x = [theta0;0];
30 for k = 1:n
31     torque = -inr*omegaN*(2*x(2) + omegaN*x(1));
32     xP2(:,k) = [x;torque];
33     x = RK4(@RHS,x,dT,0,torque,inr);
34 end
35 xP2(:,k+1) = [x;torque];
36
37 t = (0:n)*dT;
38 yL = {'\theta', '\omega', 'T'};
39 lg = {'Bang-Bang' 'PD'};
40 kP = {[1 4] [2 5] [3 6]};
41 s = sprintf('PD vs. Bang-Bang omegaN = %6.3f rad/s', omegaN);
42 TimeHistory(t,[xP1;xP2],yL,s,kP,{lg lg lg});
43
44 function xDot = RHS(x,~,t,inr)
45
46 xDot = [x(2);t/inr];
47
48 end
49
50 function omegaN = FindOmegaN(r,tau)
51
52 myfun = @(x,tau,r) r - exp(-x*tau)*(1+x*tau);
53 omegaN = fzero(@(x) myfun(x,tau,r), [0, 1]);
54 end

```



Example 12.16: Torque comparison for a 90-degree rotational maneuver.

ple, a sine/cosine series is

$$\frac{dh_w}{dt} = a_0 + \sum_{k=1}^N (a_k \sin(k\omega_o t) + b_k \cos(k\omega_o t)) \quad (12.148)$$

where ω_o is the orbit rate. This is for a planet pointing satellite. The momentum stored is

$$h_w = h_w(0) + a_0 t - \frac{1}{\omega_o} \sum_{k=1}^N \frac{1}{k} (a_k \cos(k\omega_o t) - b_k \sin(k\omega_o t)) \quad (12.149)$$

The momentum wheel experiences a steady increase due to a_0 . The harmonics will cause periodic peaks. This means unloading will need to be done earlier than what would be calculated by just considering $a_0 t$. Maneuver planning must include any momentum already stored in the momentum-exchange device, or any wheel-speed biasing that may have been done.

The magnitude of the disturbances is typically on the order of nano-N. For microthrusters that operated continuously, the thruster must produce a torque greater than the disturbance. For larger thrusters, such as typical hydrazine thrusters, it determines the minimum pulsewidth and the control period. Assume that the disturbance is the constant a_0 given above. If the thruster torque is T then the minimum pulsewidth must be

$$\tau_m = \frac{a_0}{T} \tau \quad (12.150)$$

where τ is the pulsing period. The achievable τ_m will be determined by the valve dynamics.

The torque from a magnetic torquer is

$$T = M^\times B \quad (12.151)$$

where M is the dipole moment and B is the magnetic field in the body frame. A magnetic control system cannot produce a torque along the direction of B . Consequently, it is not possible to always instantaneously cancel a disturbance torque in all three axes. The sizing of the magnetic torquers is very dependent on the control algorithm. It is also very dependent on the knowledge of the planet's magnetic field. The field is dependent on the spacecraft's orbital location above the planet, the angle between the spacecraft position vector and the Sun vector, and the solar activity. The magnetic field of a planet also changes slowly with time. This complicates the prediction of the field. A magnetometer could measure the field, and the previous orbits measurements used to improve future predictions.

Sizing of the torquers can be done by taking the total momentum growth over an orbit, h , and computing the minimum dipole needed to cancel it. That is

$$h = \int_0^\tau M^\times B dt \quad (12.152)$$

where τ is the orbital period. This can be done by minimizing this cost functional over one or more orbits

$$J = (T_d - M^\times B)^T (T_d - M^\times B) + M^T M \quad (12.153)$$

The first term minimizes the quadratic torque error. The second minimizes the magnitude of the dipole. The solution is

$$S = \int_0^\tau B^\times B^\times dt \quad (12.154)$$

$$\lambda = S^{-1}h \quad (12.155)$$

$$M(t) = \lambda^\times B(t) \quad (12.156)$$

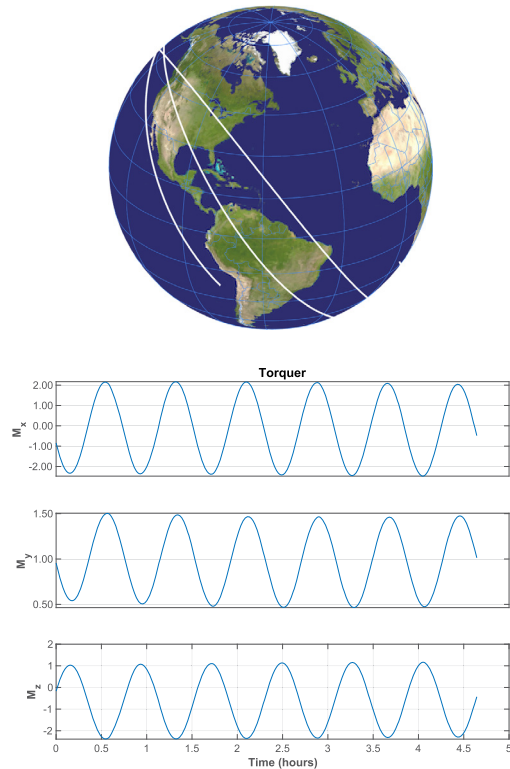
This weights the torque error and control-dipole magnitude the same.

Example 12.17 computes the optimal magnetic dipole over three ISS orbits assuming a constant amount of momentum that needs to be removed. This is equivalent to a constant disturbance torque. The example algorithm works in the inertial frame. The integrated torque exactly matches the desired value.

```

1 %% Compute the optimal magnetic dipole
2
3 tD      = [1;2;3]*1e-6;
4 n       = 1000;
5 [e1, jD0] = ISSOrbit;
6 p       = Period(e1(1));
7 tau     = 3*p;
8 t       = linspace(0,tau,n);
9 r       = RVOrbGen(e1,t);
10
11 PlotOrbit(r,t,jD0)
12 jD      = jD0 + t/86400;
13 b       = BDipole(r,jD);
14 s       = zeros(3,3);
15 for k = 1:n
16     s = s + SkewSq(b(:,k));
17 end
18
19 h       = tD*tau;
20 lambda  = s\h;
21
22 m       = zeros(3,n);
23 tC      = zeros(3,1);
24 for k = 1:n
25     m(:,k) = cross(lambda,b(:,k));
26     tC     = tC + cross(m(:,k),b(:,k));
27 end
28 tC/tau
29 TimeHistory(t,m,{ 'M_x' 'M_y' 'M_z' }, 'Torquer')

```



Example 12.17: The optimal magnetic dipole to cancel a constant disturbance torque.

References

- [1] M. Bagheri, M. Kabganian, R. Nadafi, Three-axis attitude control design for a spacecraft based on Lyapunov stability criteria, *Scientia Iranica* 20 (4) (2013) 1302–1309.
- [2] D.A. Vallado, *Fundamentals of Astrodynamics and Applications*, 2nd ed., McGraw-Hill, 1997.
- [3] SSC, PRISMA, <http://www.ssc.se/?id=7611>.